ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

CS315 — MÁY HỌC NÂNG CAO

Giảng viên hướng dẫn: ThS. Đặng Việt Dũng

SỔ TAY BÀN GIAO

Trợ lý AI phát hiện và định vị nốt phổi trên X-quang ngực:
tinh chỉnh LoRA mô hình ngôn ngữ–thị giác cỡ nhỏ

> **Sổ tay này dành cho ai?** Cho nhóm sinh viên nhận tiếp đồ án. Bạn chỉ cần biết Python và máy học cơ bản. Bạn chưa cần biết VLM, LoRA, Colab GPU hay ảnh y khoa: sổ tay giải thích từ đầu.
>
> **Trạng thái lúc bàn giao (2026-11-13):**
>
> - Pipeline đã viết xong. 75 kiểm thử trên CPU qua (`pytest -q` trong `code/`).
> - Phần chạy GPU (transformers, peft, ultralytics, torchxrayvision) mới chỉ được kiểm tra cú pháp. Phiên GPU đầu tiên **phải** chạy 7 mục "Kiểm tra bắt buộc trên Colab" trong RUNBOOK.
> - Chưa tải bộ dữ liệu nào. Chưa chạy thí nghiệm nào. Báo cáo còn các ô đỏ [–].

Repo:	`github.com/minhdang900/uit-cs315-lung-nodule-cxr-vlm` (private)
Tag bàn giao:	`v0.1.0-handoff`
Tài liệu gốc:	`docs/design/2026-11-12-lung-nodule-cxr-vlm-lora-design.md` (spec)
Lệnh chạy:	`code/RUNBOOK.md`

TP. Hồ Chí Minh, tháng 11 năm 2026

Mục lục

1 Đọc gì trước, làm gì trước **3**
1.1 Danh sách việc khởi động (một trang) . . . 3
1.2 Thứ tự đọc tài liệu . . . 3

2 Đề tài trong một trang **4**
2.1 Bài toán . . . 4
2.2 Đồ án làm gì . . . 4
2.3 Câu hỏi nghiên cứu và tiêu chí thành công . . . 5

3 Kiến thức nền **5**
3.1 X-quang ngực, nốt phổi và khối . . . 5
3.2 CNN phân loại và CNN phát hiện . . . 6
3.3 Mô hình ngôn ngữ–thị giác (VLM) . . . 6
3.4 Tinh chỉnh toàn phần và LoRA . . . 7
3.5 Tinh chỉnh có giám sát (SFT) và che loss . . . 7
3.6 Ma trận nhầm lẫn: sensitivity, specificity, PPV, NPV . . . 8
3.7 Đường ROC, AUC và "sensitivity ở specificity 0,90" . . . 9
3.8 Chọn ngưỡng trên tập validation . . . 9
3.9 IoU: box dự đoán trùng box thật bao nhiêu . . . 10
3.10 FROC: sensitivity theo số báo nhầm mỗi ảnh . . . 10
3.11 Hiệu chỉnh xác suất: ECE và Brier . . . 10
3.12 Thống kê: bootstrap, McNemar, DeLong, Holm . . . 11
3.13 Kiểm toán che vùng T1/T2/T3 và Δ_{occ} . . . 12
3.14 Quá khớp, seed và phương sai . . . 12
3.15 Rò rỉ dữ liệu và chia theo bệnh nhân . . . 13

4 Kiến trúc hệ thống **14**
4.1 Bức tranh tổng thể . . . 14
4.2 Cấu trúc repo . . . 14
4.3 Bản đồ code . . . 14
4.4 Luồng file . . . 16

5 Các cấu hình và thí nghiệm **17**
5.1 Bảy cấu hình . . . 17
5.2 Sáu thí nghiệm E0–E5 . . . 18
5.3 Lịch đề xuất . . . 18

6 Runbook từng bước **19**
6.1 Môi trường trên máy cá nhân (CPU) . . . 19

6.2 Clone repo vào Google Drive . . . 19
6.3 Quyền tải MedGemma và HF token . . . 20
6.4 Colab và GPU . . . 20
6.5 Tải dữ liệu . . . 21
6.6 Chuẩn bị dữ liệu (phần đầu E0) . . . 23
6.7 Mỗi phiên Colab (RUNBOOK §0) . . . 24
6.8 Bảy kiểm tra bắt buộc (phiên GPU đầu tiên) . . . 24
6.9 Chạy các thí nghiệm . . . 26
6.10 Ngân sách và cắt giảm . . . 29
6.11 Lỗi thường gặp . . . 29
6.12 Điền báo cáo từ kết quả . . . 30

7 Quy trình làm việc nhóm với Git/GitHub 31
7.1 Mô hình nhánh (git-flow rút gọn) . . . 31
7.2 Các lệnh từng bước . . . 32
7.3 Quy ước commit . . . 32
7.4 Những thứ không bao giờ commit . . . 33
7.5 Ghi lại quyết định . . . 33

8 Quy tắc bắt buộc và tiêu chí hoàn thành 34
8.1 Nên và không nên . . . 34
8.2 Định nghĩa “xong” cho từng thí nghiệm . . . 34

9 Hỏi đáp thường gặp 35

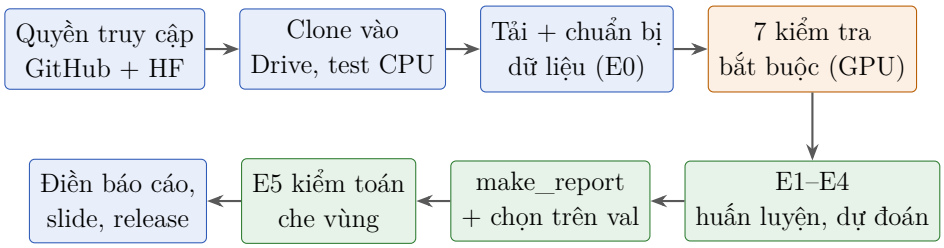
A Bảng thuật ngữ Việt–Anh 36

1 Đọc gì trước, làm gì trước

1.1 Danh sách việc khởi động (một trang)

Làm theo thứ tự. Mỗi dòng có chỗ đánh dấu. Cột cuối chỉ chỗ đọc chi tiết.

	#	Việc	Xem
☐	1	Mở repo public `minhdang900/uit-cs315-lung-nodule-cxr-vlm` trên GitHub; đọc bảng "Trạng thái và việc cần làm" đầu `code/RUNBOOK.md`. Muốn push thì xin quyền collaborator.	§7
☐	2	Clone repo một lần vào `/content/drive/MyDrive` (RUNBOOK §0, bước "Lần đầu tiên"). Clone chỗ khác thì đổi `REPO`.	§6.2
☐	3	Trên máy cá nhân: tạo venv, chạy `pip install -e ".[dev]"` rồi `pytest -q`. Phải thấy 79 passed (cần có torch; nếu không, vài test bị skip).	§6.1
☐	4	Xin quyền tải `google/medgemma-4b-it` trên Hugging Face (chấp nhận điều khoản Health AI Developer Foundations). Tạo HF token. Lưu vào Colab Secrets tên `HF_TOKEN`.	§6.3
☐	5	Mua/chuẩn bị Colab Pro (GPU L4 hoặc A100). Không dùng T4.	§6.4
☐	6	Đọc điều khoản 3 bộ dữ liệu. E0 **đã xong**: xin GVHD chia sẻ thư mục dữ liệu trên Drive, tạo lối tắt `cs315-data` trong `My Drive`.	§6.2, §6.5
☐	7	Chỉ khi không được chia sẻ: tự tải dữ liệu và chạy 3 script `prepare_*`. Mọi người chạy `data_stats.py` để có `$O/e0_data_edges.json`.	§6.6
☐	8	Phiên GPU đầu tiên: chạy đủ 7 mục kiểm tra bắt buộc. Mục nào hỏng thì dừng.	§6.8
☐	9	Chạy E0 → E5 theo đúng thứ tự. Ghi ngân sách vào `outputs/budget.md`.	§6.9
☐	10	Chạy `make_report.py`, điền các ô đỏ trong báo cáo.	§6.12



Hình 1. Lộ trình tổng quát. Hộp cam là cổng bắt buộc: chưa qua thì không chạy tiếp.

1.2 Thứ tự đọc tài liệu

1. **Sổ tay này**, §1–§4. Mục đích: hiểu bài toán và các khái niệm. Khoảng 2–3 giờ.
2. **Spec** `docs/design/2026-11-12-lung-nodule-cxr-vlm-lora-design.md`. Đây là văn bản có hiệu lực cao nhất. Đọc kỹ §5 (phương pháp), §7 (thí nghiệm), §8 (đánh giá) và **§8.5 (ghi chú triển khai)**. Khi sổ tay và spec mâu thuẫn, **spec đúng**.
3. **RUNBOOK** `code/RUNBOOK.md`. Đây là nguồn duy nhất cho lệnh chạy. Sổ tay chép lệnh từ đây và giải thích thêm.
4. **Báo cáo** `report/main.tex` (PDF: `report/main.pdf`). Đọc lướt chương 1–3 để biết văn phong học thuật. Chương 4b là nơi điền kết quả.
5. **Nhật ký quyết định** `docs/decisions.md`. Biết vì sao mọi thứ đang như vậy, trước khi định đổi.

6. **Tổng quan tài liệu** `docs/lit-review/lung-cancer-cxr/`. Chỉ đọc khi cần trích dẫn. Bắt đầu từ `00-review-report.md`. File PDF bài báo không có trong git (bản quyền).

> **Ba quy tắc quan trọng nhất.** (1) Không bao giờ chọn cấu hình hay ngưỡng bằng số trên tập test. (2) Không commit dữ liệu, trọng số mô hình, thư mục `outputs/`, token, hay PDF có bản quyền. (3) Không tự nghĩ ra cờ dòng lệnh: chép từ RUNBOOK, hoặc đọc `argparse` trong `code/scripts/`.

2 Đề tài trong một trang

2.1 Bài toán

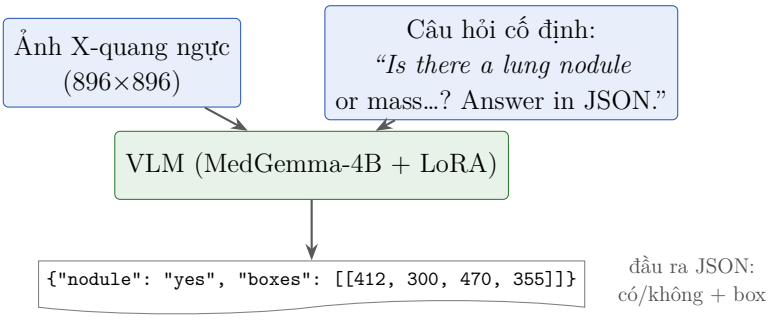
Ung thư phổi là nguyên nhân tử vong do ung thư hàng đầu. Nhiều ca lần đầu lộ ra dưới dạng một **nốt** hoặc **khối** trên X-quang ngực. Nốt nhỏ và mờ rất dễ bị bỏ sót.

Mô hình ngôn ngữ–thị giác (*vision-language model, VLM*) là mô hình nhận ảnh và câu hỏi, rồi trả lời bằng chữ. Người ta đang giới thiệu VLM như "trợ lý đọc phim". Nhưng các đánh giá độc lập năm 2026 cho thấy VLM gần như **không thấy nốt phổi**. Ví dụ, MedGemma-4B chỉ phát hiện 5,2% nốt trên bộ JSRT, dù không báo nhầm ca nào (specificity 100%). Trong khi đó, phần mềm CNN chuyên dụng đạt AUROC 0,92–0,99. Kiểu sai "luôn nói bình thường" này gọi là **chế độ báo bình thường** (*normal-mode failure*).

2.2 Đồ án làm gì

Đồ án xây một nguyên mẫu trợ lý AI. Trợ lý trả lời: ảnh có nốt/khối không, và nếu có thì ở đâu (bounding box). Có ba phần việc:

1. **Benchmark**: so hai VLM nhỏ (MedGemma-4B, Qwen2.5-VL-3B) với hai CNN (DenseNet-121, YOLOv8) trên cùng cách chia dữ liệu, trên ba bộ: VinBigData, NIH ChestX-ray14, JSRT.
2. **Tinh chỉnh LoRA** MedGemma-4B: chỉ phân loại; phân loại kèm box; có và không cân bằng lớp.
3. **Kiểm toán "nhìn đúng chỗ"**: đo IoU, và thử che nốt so với che một vùng ngẫu nhiên cùng diện tích.



Hình 2. Trợ lý nhận ảnh và một câu hỏi cố định, trả lời bằng JSON.

2.3 Câu hỏi nghiên cứu và tiêu chí thành công

Q1. LoRA có đưa sensitivity của VLM, đo ở specificity 0,90, lên **ít nhất 0,60** trên tập test VinBigData (trong phân phối) không?
Q2. Cải thiện đó có còn khi kiểm tra ngoài phân phối (NIH, JSRT) và khi che nốt không?
Tiêu chí thành công (spec §2): có bảng so sánh 7 cấu hình × 3 tập test, kèm khoảng tin cậy 95%, và kết luận được Q1, Q2 *dù theo hướng nào*. Kết quả âm tính (LoRA không đạt 0,60) vẫn là kết quả hợp lệ.

Các khái niệm "sensitivity ở specificity 0,90", "trong/ngoài phân phối", "khoảng tin cậy" được giải thích ở §3.

Không viết "ngang bác sĩ". Ngưỡng 0,60 nằm cùng khoảng với sensitivity của người đọc trong vài nghiên cứu, nhưng các nghiên cứu đó không đo ở specificity 0,90. Spec §2 cấm viết "ngang bác sĩ". Đây là **nguyên mẫu nghiên cứu**, không phải công cụ lâm sàng.

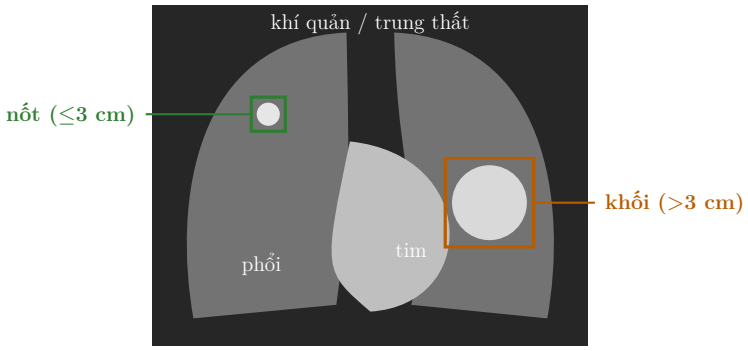
3 Kiến thức nền

Chương này giải thích mọi khái niệm bạn cần trước khi chạy lệnh. Mỗi mục trả lời ba câu: nó là gì, vì sao đồ án cần nó, và trong code nó nằm ở đâu.

3.1 X-quang ngực, nốt phổi và khối

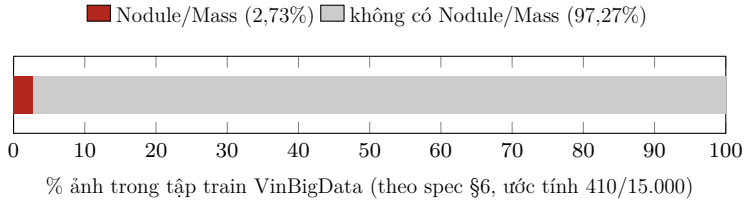
X-quang ngực (*chest X-ray, CXR*) là ảnh xám 2 chiều. Tia X đi qua lồng ngực. Mô đặc (xương, tim, khối u) cản tia nhiều nên hiện màu sáng. Phổi chứa khí nên hiện màu tối.

- **Nốt phổi** (*lung nodule*): đốm tròn, đường kính ≤ 3 cm.
- **Khối** (*mass*): tổn thương > 3 cm.
- Đồ án **gộp** hai loại thành một lớp dương "Nodule/Mass" (quyết định ngày 2026-11-12).
- **Độ mờ** (*subtlety*): nốt khó thấy đến mức nào. Bộ JSRT chấm từ 1 (rất khó thấy) đến 5 (rõ).



Hình 3. Sơ đồ X-quang ngực (minh họa, không phải ảnh thật). Khung màu là bounding box, tức nhãn vị trí.

Vì sao khó? Nốt nhỏ có thể chỉ chiếm vài chục pixel trên ảnh 896×896. Nốt có thể nằm sau xương sườn hoặc tim. Và ca dương rất hiếm: trong tập train VinBigData chỉ khoảng 2,73% ảnh có Nodule/Mass (spec §6).

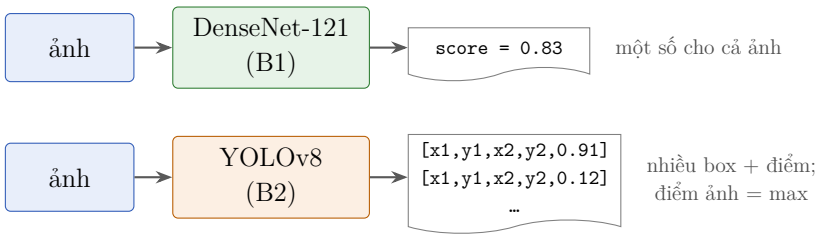


Hình 4. Mất cân bằng lớp (*class imbalance*): ca dương rất hiếm.

3.2 CNN phân loại và CNN phát hiện

Mạng tích chập (*convolutional neural network, CNN*) là mạng học đặc trưng ảnh bằng các bộ lọc trượt. Đồ án dùng hai baseline CNN:

- **B1 – bộ phân loại** (*classifier*): DenseNet-121 đã tiền huấn luyện trên X-quang (thư viện `torchxrayvision`). Đầu ra là *một* xác suất cho cả ảnh: "có nốt" bao nhiêu phần trăm. Không có vị trí.
- **B2 – bộ phát hiện** (*detector*): YOLOv8. Đầu ra là *nhiều* box, mỗi box có điểm tin cậy. Điểm của ảnh = điểm box cao nhất (`baselines/yolo.py`).

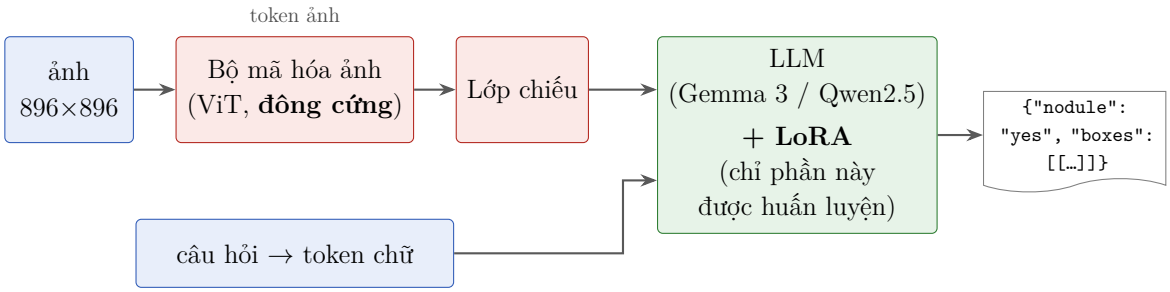


Hình 5. Bộ phân loại cho một xác suất; bộ phát hiện cho danh sách box.

3.3 Mô hình ngôn ngữ–thị giác (VLM)

Mô hình ngôn ngữ lớn (*large language model, LLM*) đoán từ tiếp theo dựa trên các từ trước. Văn bản được cắt thành **token** (mẩu từ). **VLM** gắn thêm một **bộ mã hóa ảnh** (*image encoder*) vào LLM:

1. Bộ mã hóa ảnh (thường là ViT, (*Vision Transformer*)) biến ảnh thành một dãy vector. MedGemma dùng MedSigLIP; ảnh luôn được đưa về 896×896.
2. Một **lớp chiếu** (*projector*) đổi các vector đó về cùng "ngôn ngữ" với token chữ.
3. LLM đọc token ảnh + token câu hỏi, rồi sinh câu trả lời từng token một.



Hình 6. Cấu trúc VLM. Trong đồ án, bộ mã hóa ảnh và lớp chiếu bị đông cứng; LoRA chỉ gắn vào các lớp attention và MLP của LLM (xem `LORA_TARGET_REGEX` trong `train/sft_data.py`).

Hai VLM trong đồ án:

- **MedGemma-4B-it** (`google/medgemma-4b-it`): mô hình y khoa của Google. Là VLM chính. Phải chấp nhận điều khoản trên Hugging Face.
- **Qwen2.5-VL-3B-Instruct** (`Qwen/Qwen2.5-VL-3B-Instruct`): mô hình đa năng, vốn biết xuất tọa độ box. Dùng để so "y khoa vs đa năng".

Điểm liên tục từ VLM. VLM trả lời bằng chữ, nhưng để vẽ ROC ta cần một *điểm số* liên tục. Code (`vlm/hf.py`, hàm `yes_probability`) làm như sau: ép mô hình đọc hai câu trả lời mở đầu `{"nodule": "yes` và `{"nodule": "no`, tính log-xác suất của phần khác nhau, rồi chuẩn hóa thành $P(\text{yes}) \in [0, 1]$ (spec §8.5).

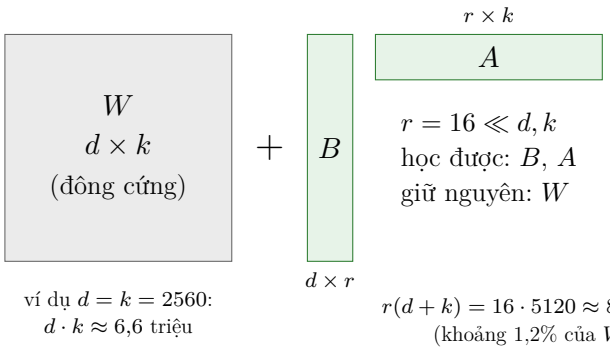
3.4 Tinh chỉnh toàn phần và LoRA

Tinh chỉnh (*fine-tuning*) là huấn luyện tiếp một mô hình đã có trên dữ liệu mới. **Tinh chỉnh toàn phần** (*full fine-tuning*) cập nhật mọi trọng số. Với mô hình 4 tỷ tham số, cách này cần rất nhiều bộ nhớ GPU.

LoRA (*Low-Rank Adaptation, thích ứng hạng thấp*) giữ nguyên ma trận trọng số gốc W (kích thước $d \times k$) và học thêm hai ma trận nhỏ B ($d \times r$) và A ($r \times k$), với r rất nhỏ:

$$W' = W + \frac{\alpha}{r} BA.$$

Đồ án dùng $r = 16$, $\alpha = 32$, dropout 0,05 (`train/lora.py`). Số tham số học chỉ là $r(d + k)$ thay vì $d \cdot k$.



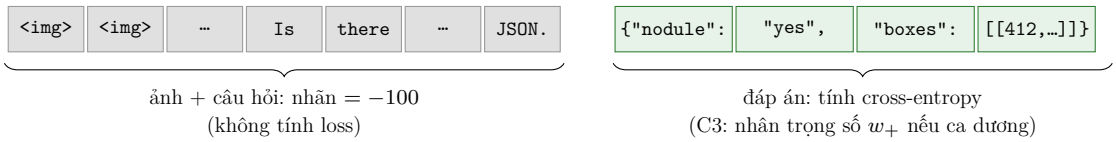
Hình 7. LoRA: thêm tích BA hạng thấp vào ma trận trọng số đông cứng W. Con số d chỉ để minh họa.

Sau huấn luyện, chỉ lưu A, B (gọi là **adapter**) vào thư mục `final/`. Lúc dự đoán, ta tải mô hình gốc rồi gắn adapter (cờ `--adapter`).

3.5 Tinh chỉnh có giám sát (SFT) và che loss

SFT (*supervised fine-tuning*) dạy mô hình bằng cặp (câu hỏi, đáp án mẫu). Mô hình học dự đoán từng token của đáp án. Loss là **cross-entropy** (*entropy chéo*) trên từng token.

Che loss (*loss masking*): ta chỉ muốn mô hình học *trả lời*, không học lặp lại câu hỏi. Vì vậy mọi token của phần câu hỏi (và token ảnh) có nhãn -100, nghĩa là "bỏ qua khi tính loss" (`build_example` trong `train/sft_data.py`).



Hình 8. Che loss: chỉ các token của đáp án được tính loss.

Cân bằng lớp trong C3. Ca dương hiếm, nên mô hình dễ học cách "luôn nói no". C3 dùng hai cách chống lại:

1. **Oversampling** (*lấy mẫu lặp*): lặp lại ca dương cho đến khi tỷ lệ dương:âm là 1:1 (`oversample_indices`, cờ `--balance`).
2. **Trọng số loss**: nhân loss của *mọi* token đáp án ở ca dương với w_+ (cờ `--pos-weight 2.0` trong RUNBOOK; giá trị cố định, **không** tinh chỉnh trên val). Loss được chia cho tổng số token nhãn của cả cửa sổ tích lũy gradient (spec §8.5).

Tích lũy gradient (*gradient accumulation*): GPU chỉ chứa được 1 ảnh mỗi lần (batch = 1). Ta cộng gradient của 8 ảnh rồi mới cập nhật trọng số một lần (`--grad-accum 8`). Kết quả gần giống batch = 8.

3.6 Ma trận nhầm lẫn: sensitivity, specificity, PPV, NPV

Khi đã chọn một **ngưỡng** (*threshold*) t, mỗi ảnh có điểm $s \geq t$ được đoán "có nốt". So với nhãn thật, ta có bốn ô:

Thật \ Mô hình	Mô hình: có nốt	Mô hình: không
Thật: có	**TP** dương thật (bắt đúng nốt)	**FN** âm giả (**bỏ sót** nốt)
Thật: không	**FP** dương giả (báo nhầm)	**TN** âm thật (đúng là không)

$$\text{Sensitivity} = \frac{TP}{TP+FN}$$

$$\text{Specificity} = \frac{TN}{TN+FP}$$

$$\text{PPV} = \frac{TP}{TP+FP} \qquad \text{NPV} = \frac{TN}{TN+FN}$$

Hình 9. Ma trận nhầm lẫn (*confusion matrix*) và bốn chỉ số cơ bản (`binary_metrics` trong `metrics/detection.py`).

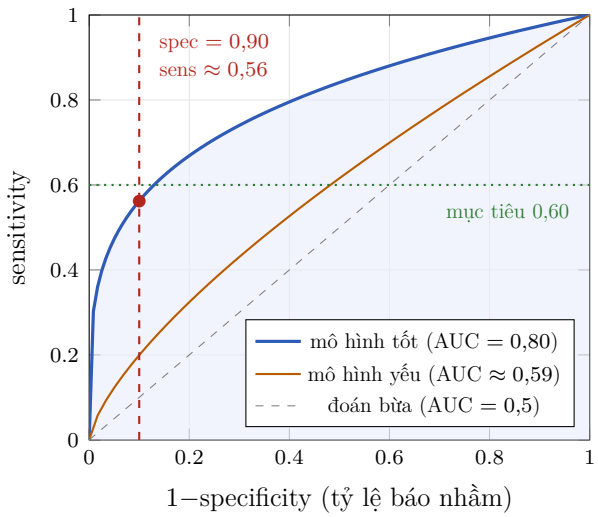
- **Sensitivity** (*độ nhạy, recall*): trong các ca có nốt, mô hình bắt được bao nhiêu.
- **Specificity** (*độ đặc hiệu*): trong các ca không có nốt, mô hình nói đúng "không" bao nhiêu.
- **PPV** (*positive predictive value, precision*): khi mô hình nói "có", đúng bao nhiêu. PPV phụ thuộc mạnh vào **tỷ lệ bệnh** (*prevalence*). Vì vậy code báo thêm `ppv_prev_1pct`, `ppv_prev_3pct`: PPV quy về tỷ lệ bệnh 1% và 3%. Trên JSRT (62% ảnh có nốt), PPV không có ý nghĩa lâm sàng.
- **NPV**: khi mô hình nói "không", đúng bao nhiêu.

Chế độ báo bình thường thể hiện bằng: specificity ≈ 1, sensitivity ≈ 0. Code báo thêm `no_on_pos` (tỷ lệ trả lời "no" trên ca dương) và `yes_on_neg`.

3.7 Đường ROC, AUC và "sensitivity ở specificity 0,90"

Nếu thay đổi ngưỡng t từ cao xuống thấp, ta được nhiều cặp (sensitivity, 1 − specificity). Nối lại thành **đường ROC** (*receiver operating characteristic*). **AUC-ROC** (*area under the curve*) là diện tích dưới đường: 1,0 là hoàn hảo, 0,5 là đoán bừa.

Chỉ số chính của đồ án: sensitivity tại điểm có specificity 0,90, tức 1 − spec = 0,10. Ý nghĩa: "nếu chỉ cho phép báo nhầm 10% ca lành, mô hình bắt được bao nhiêu ca bệnh?".



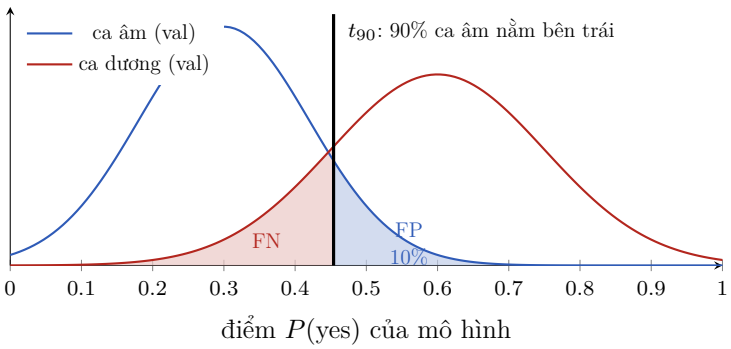
Hình 10. Đường ROC minh họa (số liệu giả, chỉ để giải thích). Điểm đỏ là chỉ số chính. Mô hình xanh có AUC 0,80 nhưng vẫn chưa đạt 0,60 ở spec 0,90: AUC cao không đảm bảo đạt mục tiêu.

AUC-PR (*diện tích dưới đường precision–recall*): khi lớp dương hiếm, AUC-PR nhạy hơn AUC-ROC. Code báo cả hai (`score_metrics`).

3.8 Chọn ngưỡng trên tập validation

Dữ liệu được chia ba phần: **train** (học), **validation** (chọn ngưỡng, chọn cấu hình), **test** (chấm điểm cuối, dùng một lần).

Ngưỡng t_{90} được chọn **trên validation**: lấy ngưỡng nhỏ nhất sao cho ít nhất 90% ca âm trên val có điểm dưới ngưỡng (`threshold_at_specificity`). Sau đó áp đúng t_{90} này lên các tập test. Vì vậy specificity trên test có thể khác 0,90 một chút. Đó là điều bình thường và trung thực.



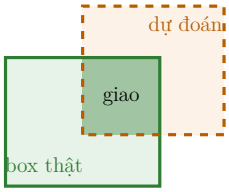
Hình 11. Chọn ngưỡng t_{90} trên validation (minh họa). Phần xanh tô là 10% ca âm bị báo nhầm; phần đỏ tô là ca dương bị bỏ sót.

Các ngưỡng được lưu trong `outputs/report/thresholds.json`, khóa có dạng `config|seed|run`, ví

dụ `"C3|0|0"`. Mỗi khóa có `t90` và `t95`.

3.9 IoU: box dự đoán trùng box thật bao nhiêu

IoU (*Intersection over Union*) $= \dfrac{\text{diện tích giao}}{\text{diện tích hợp}}$. IoU = 1 là trùng khít; 0 là không chạm nhau.



$$\text{IoU} = \frac{\text{giao}}{\text{thật} + \text{dự đoán} - \text{giao}}$$

$$\Rightarrow \quad \text{ví dụ: } \frac{1{,}44}{4{,}8 + 4{,}4 - 1{,}44} \approx 0{,}19 \quad (\text{trượt ở ngưỡng } 0{,}3)$$

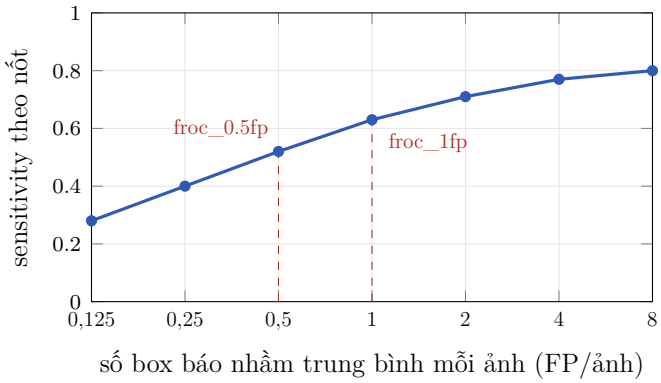
Hình 12. IoU (`metrics/localization.py`). Đồ án coi là "trúng" khi IoU $\geq 0{,}3$ (nốt nhỏ), và báo thêm ở 0,5.

Theo spec §8.5:

- `hit_iou30`, `hit_iou50`: tỷ lệ ca dương mà mô hình vừa nói "có" (điểm $\geq t_{90}$) *vừa* có box IoU $\geq 0{,}3$ (hoặc 0,5). Chỉ dùng box có điểm $\geq t_{90}$.
- **JSRT** chỉ có tâm và đường kính nốt. "Trúng" khi tâm box dự đoán (box điểm cao nhất) nằm trong bán kính nốt cộng biên 5 mm. Code đổi pixel ra mm bằng $0{,}175 \times 2048/896 = 0{,}4$ mm/px, nên 5 mm = 12,5 px.

3.10 FROC: sensitivity theo số báo nhầm mỗi ảnh

Bộ phát hiện có thể đưa ra nhiều box trên một ảnh. **FROC** (*free-response ROC*) vẽ: trục ngang là *số box báo nhầm trung bình mỗi ảnh* (FP/ảnh), trục dọc là tỷ lệ nốt thật được một box trúng (IoU $\geq 0{,}3$). Code báo hai điểm `froc_0.5fp` và `froc_1fp`. FROC dùng *mọi* box, không lọc theo ngưỡng.

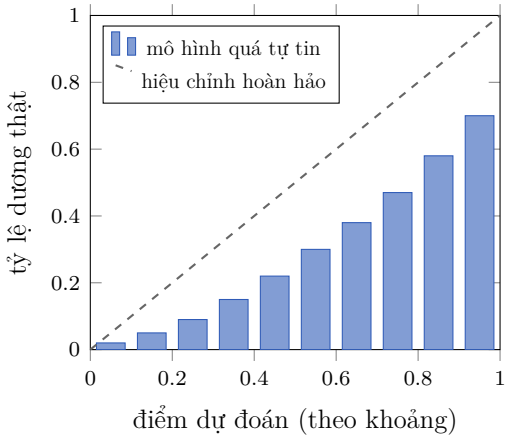


Hình 13. Đường FROC minh họa (số liệu giả).

3.11 Hiệu chỉnh xác suất: ECE và Brier

Một mô hình **được hiệu chỉnh tốt** (*well-calibrated*) khi nó nói "70%" thì đúng khoảng 70% số lần.

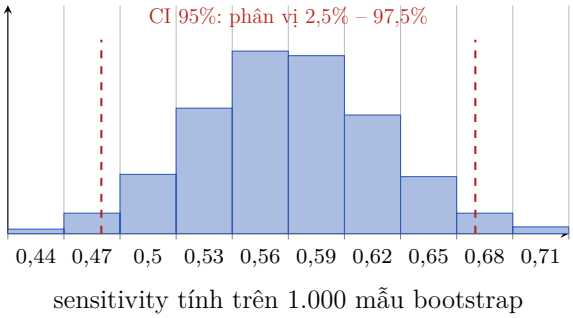
- **ECE** (*expected calibration error*): chia điểm vào 10 khoảng, đo chênh lệch giữa điểm trung bình và tỷ lệ dương thật trong mỗi khoảng, rồi lấy trung bình có trọng số. Càng nhỏ càng tốt.
- **Brier score**: trung bình $(s - y)^2$. **Balanced Brier**: trung bình của Brier trên ca dương và Brier trên ca âm, để lớp hiếm không bị lấn át.



Hình 14. Biểu đồ độ tin cậy (*reliability diagram*) minh họa. Cột thấp hơn đường chéo nghĩa là mô hình tự tin quá mức.

3.12 Thống kê: bootstrap, McNemar, DeLong, Holm

Tập test nhỏ (ví dụ chỉ khoảng 82 ảnh dương trên VinBigData test, theo ước tính trong spec §6). Một con số sensitivity duy nhất có thể dao động nhiều. Thống kê giúp trả lời: "chênh lệch này có thật, hay do may rủi?".



Hình 15. Bootstrap minh họa: lấy mẫu lại có hoàn lại nhiều lần, tính chỉ số mỗi lần; khoảng giữa phân vị 2,5% và 97,5% là CI 95%.

Bootstrap CI (*khoảng tin cậy bootstrap*)
Rút ngẫu nhiên có hoàn lại cùng số ảnh dương và cùng số ảnh âm (phân tầng theo nhãn), tính lại chỉ số. Lặp 1.000 lần (`n_boot=1000` trong `eval/report.py`). Lấy phân vị 2,5% và 97,5%. *Hạn chế*: lấy mẫu theo ảnh, không theo bệnh nhân, nên CI có thể hẹp hơn thực tế (spec §8.5).

McNemar
So *hai mô hình trên cùng một tập ảnh* cho sensitivity (hoặc specificity). Chỉ đếm các ảnh mà hai mô hình bất đồng: A đúng B sai, và A sai B đúng. Nếu hai số này chênh nhiều, khác biệt có ý nghĩa. Code dùng bản chính xác (*exact*).

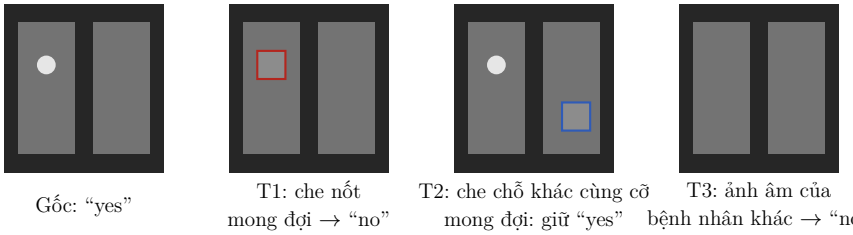
DeLong
So *hai AUC* trên cùng tập ảnh, có tính tới việc hai mô hình chấm cùng ảnh nên điểm của chúng tương quan.

Holm
Khi so nhiều cặp, xác suất "ăn may" ít nhất một lần tăng lên. Holm điều chỉnh p-value: sắp xếp p từ nhỏ đến lớn, nhân p nhỏ nhất với số phép so, p kế tiếp với (số phép so -1), v.v. Cột `*_holm` trong `comparisons.csv`.

Quy tắc đọc p-value. $p < 0{,}05$ (sau Holm) mới được viết "khác biệt có ý nghĩa thống kê". p lớn *không* chứng minh hai mô hình bằng nhau; nó chỉ nói dữ liệu chưa đủ để phân biệt.

3.13 Kiểm toán che vùng T1/T2/T3 và Δ_{occ}

Mô hình có thể đoán đúng "có nốt" vì lý do sai, ví dụ nhìn thấy dây điện cực hay chữ đánh dấu trên phim. Phép thử che vùng (*occlusion test*) kiểm tra điều này (`audit/occlusion.py`, `audit/run.py`, script `run_audit.py`).



Hình 16. Ba phép thử. T1: che box nới 10% bằng giá trị xám trung bình. T2: che một vùng cùng kích thước, không giao T1, $\geq 90\%$ nằm trong mặt nạ phổi. T3: thay bằng một ảnh âm ngẫu nhiên.

Tỷ lệ đổi đáp án (*flip rate*): trong các ca mô hình ban đầu nói "yes" (điểm $\geq t_{90}$), bao nhiêu % chuyển thành "no" sau khi che.

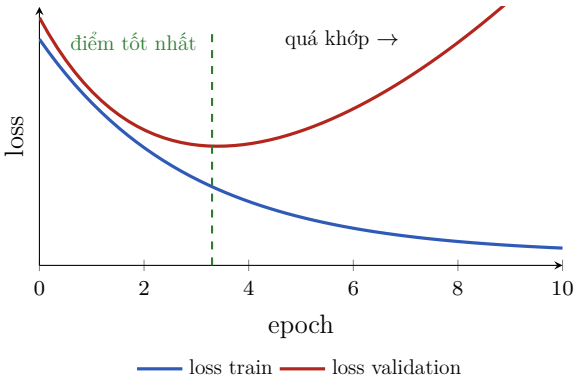
$$\Delta_{\text{occ}} = \text{flip}_{T1} - \text{flip}_{T2}.$$

Δ_{occ} lớn và dương nghĩa là mô hình thật sự dựa vào vùng nốt. $\Delta_{\text{occ}} \approx 0$ nghĩa là che chỗ nào cũng như nhau: mô hình không "nhìn" nốt. Hai vế tính trên *cùng* các ảnh tạo được T2 (`n_t2_ok`). Trong file JSON, số cần báo cáo là `delta`, kèm `n_t2_ok`. `flip_t1_all` chỉ để tham khảo.

Phân tầng (*stratification*): tính riêng theo kích thước nốt (VinBigData/NIH: tertile diện tích box, ngưỡng lấy từ ca dương train VinBigData; JSRT: ≤ 10, 10–20, > 20 mm) và theo độ mờ JSRT 1–5.

3.14 Quá khớp, seed và phương sai

Quá khớp (*overfitting*): mô hình học thuộc tập train, nên loss train giảm mãi nhưng kết quả trên validation xấu đi.



Hình 17. Quá khớp (minh họa). DenseNet B1 giữ trọng số epoch có val AUC cao nhất; YOLO B2 dùng `best.pt`.

Seed là số khởi tạo bộ sinh ngẫu nhiên. Cùng code, cùng dữ liệu, khác seed có thể cho kết quả khác. Vì vậy mỗi cấu hình chạy 3 seed (0, 1, 2) và báo **trung bình ± độ lệch chuẩn** (*mean* ± *sd*).

Seed cũng quyết định tập ảnh âm được lấy mẫu (`subsample_negatives`), thứ tự oversampling, và khởi tạo LoRA. Lưu ý: cách chia train/val/test của VinBigData cố định với `--seed 0` của `prepare_vinbig.py`; seed của thí nghiệm không đổi cách chia.

3.15 Rò rỉ dữ liệu và chia theo bệnh nhân

Rò rỉ dữ liệu (*data leakage*): thông tin từ tập test lọt vào lúc huấn luyện hoặc lúc chọn mô hình. Kết quả khi đó đẹp giả tạo. Các dạng thường gặp:

1. **Cùng bệnh nhân ở cả train và test.** Mô hình nhận ra bệnh nhân, không phải nốt. Cách tránh: chia theo bệnh nhân (*patient-level split*).
2. **Nhìn tập test để chọn.** Chọn ngưỡng, epoch, cấu hình theo số trên test. Cách tránh: chỉ chọn trên validation.
3. **Mô hình nền đã thấy dữ liệu test lúc tiền huấn luyện.** VinDr-CXR, NIH được dùng rộng rãi để huấn luyện VLM. Cách tránh: không tránh được hoàn toàn; phải nêu thành hạn chế (spec §9).
4. **Bộ dữ liệu trùng nguồn.** NODE21 trùng nguồn với JSRT và NIH, nên không dùng làm test.



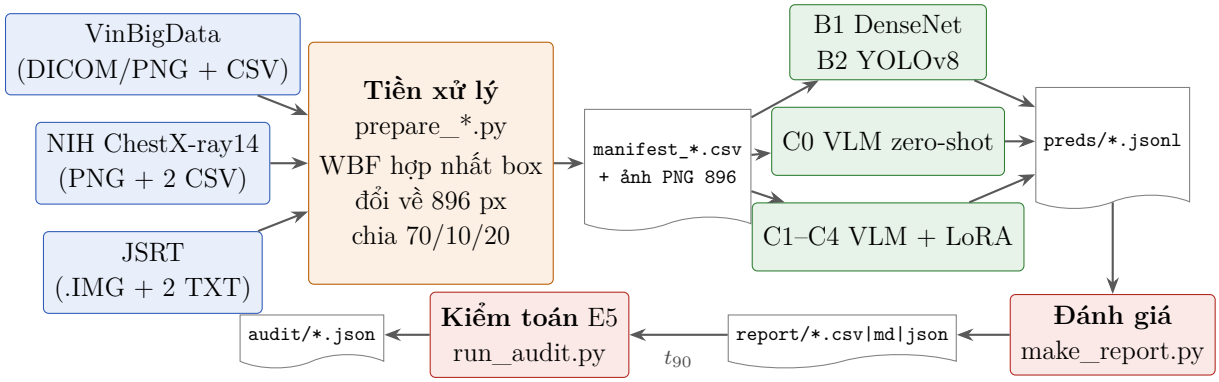
Hình 18. Mỗi chấm là một ảnh, chữ là mã bệnh nhân. Bên trái, ảnh của bệnh nhân A nằm cả ở train và test.

Tình trạng trong code (cần biết).

- **VinBigData** được chia theo `image_id` (70/10/20, phân tầng theo nhãn; spec §6). Đây **không** phải chia theo bệnh nhân: bản Kaggle không có mã bệnh nhân (cột `patient_id` trong manifest để trống). Nếu một bệnh nhân có nhiều ảnh, ảnh của họ có thể rơi vào cả train và test. Đây là **rủi ro rò rỉ**, phải nêu thành hạn chế trong báo cáo (xem D9 trong `docs/decisions.md`). Đừng mô tả cách chia này là chia theo bệnh nhân.
- **NIH**: ảnh âm được lấy từ các bệnh nhân *không* có ảnh dương (`data/nih.py`). NIH chỉ dùng làm test, nên không rò sang train.
- **JSRT**: chỉ dùng làm test.
- Bootstrap không gộp theo bệnh nhân, nên CI có thể quá hẹp. Phải nêu trong báo cáo.

4 Kiến trúc hệ thống

4.1 Bức tranh tổng thể



Hình 19. Luồng hệ thống: dữ liệu → tiền xử lý → mô hình → dự đoán → đánh giá → kiểm toán.

Ý tưởng thiết kế: **mọi mô hình ghi ra cùng một định dạng** (một dòng JSON mỗi ảnh). Nhờ vậy một script đánh giá duy nhất (`make_report.py`) chấm được cả CNN lẫn VLM theo cùng một cách.

4.2 Cấu trúc repo

`uit-cs315-lung-nodule-cxr-vlm/`	
`code/`	gói Python `nodulecxr`, scripts, tests, RUNBOOK.md
`src/nodulecxr/`	thư viện (xem Bảng 1)
`scripts/`	11 script dòng lệnh
`tests/`	79 kiểm thử CPU (pytest)
`report/`	báo cáo LaTeX: main.tex → main.pdf
`docs/design/`	spec (văn bản có hiệu lực)
`docs/handoff/`	sổ tay này + README.md
`docs/lit-review/lung-cancer-cxr/`	ghi chú tổng quan (PDF không vào git)
`docs/archive/`	kế hoạch triển khai cũ, chỉ để tra lịch sử
`docs/decisions.md`	nhật ký quyết định
`data/`	`raw/` + `processed/`, **gitignore** (trừ `data/README.md`)
`outputs/`	**gitignore**: kết quả chạy (tạo khi chạy)

Hình 20. Cấu trúc thư mục. `data/` và `outputs/` nằm ở gốc repo, ngang hàng với `code/`.

Mọi lệnh trong RUNBOOK chạy từ `code/`. Vì vậy `../data/processed` là `<repo>/data/processed` và `../outputs` là `<repo>/outputs`.

4.3 Bản đồ code

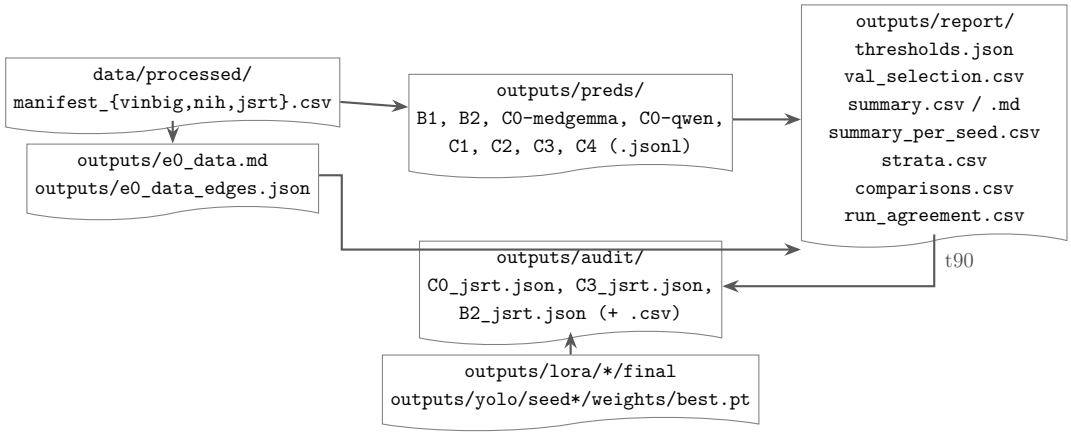
Bảng 1. Các module trong `code/src/nodulecxr/`.

Module	Làm gì	Vào → Ra
`records.py`	Định dạng manifest (12 cột), đọc/ghi JSONL, bỏ qua ảnh đã chạy (`done_ids`). Hằng `IMG_SIZE = 896`.	CSV/JSONL ↔ DataFrame
`images.py`	Đọc DICOM, đọc JSRT `.IMG`, đổi về ảnh xám 896×896.	file ảnh → PIL
`prompts.py`	Câu hỏi cố định (`QUESTION_LOC`, `QUESTION_CLS`), sinh đáp án mẫu, **parser** chịu lỗi.	text → (nhãn, box, ok)
`strata.py`	Tầng kích thước: tertile diện tích box; mm cho JSRT.	box → small/medium/large
`data/vinbig.py`	Hợp nhất box của nhiều bác sĩ bằng WBF; nhãn 1 nếu có Nodule/Mass.	train.csv → nhãn/ảnh
`data/nih.py`	Ảnh dương = có box Nodule/Mass; ảnh âm = "No Finding" của bệnh nhân khác, gấp 3 lần.	2 CSV → manifest
`data/jsrt.py`	Đọc `CLNDAT_EN.txt`, `NNDAT_EN.txt`; dựng box vuông từ tâm + đường kính.	TXT → manifest
`data/split.py`	Chia phân tầng 70/10/20; lấy mẫu ảnh âm; oversampling ca dương.	nhãn → split/chỉ số
`data/stats.py`	Bảng mô tả dữ liệu cho E0.	manifest → Markdown
`vlm/hf.py`	Tải MedGemma/Qwen từ Hugging Face, gắn adapter, sinh văn bản, tính P(yes). Chặn GPU không có bf16 thật.	ảnh + prompt → text, điểm
`vlm/chat.py`, `vlm/scoring.py`	Dựng chat template; tính log-prob của hai câu trả lời ép buộc.	—
`vlm/predict.py`	Vòng lặp dự đoán chung, sinh từng dòng JSONL.	manifest → dòng JSONL
`train/sft_data.py`	Dựng mẫu SFT có che loss; collator; dataset. Chứa `LORA_TARGET_REGEX`.	ảnh + đáp án → tensor
`train/losses.py`, `train/lora.py`	Cross-entropy có trọng số theo mẫu; cấu hình LoRA (r16, α32); Trainer con.	—
`baselines/densenet.py`	DenseNet-121 của torchxrayvision + đầu nhị phân.	ảnh 224 px → logit
`baselines/yolo.py`	Xuất dữ liệu sang định dạng YOLO; hàm chấm điểm.	manifest → data.yaml
`metrics/detection.py`	Sens/spec/PPV/NPV/F1, chọn ngưỡng, AUC, ECE, Brier, PPV theo tỷ lệ bệnh.	(y, điểm) → số
`metrics/localization.py`	IoU, khoảng cách tâm, trúng JSRT, FROC.	box → số
`metrics/stats.py`	Bootstrap CI, McNemar, DeLong, Holm.	—
`eval/report.py`	Ghép tất cả: ngưỡng trên val, chỉ số + CI trên từng tập test, phân tầng, so sánh cặp, độ ổn định.	preds + manifest → bảng
`audit/occlusion.py`, `audit/run.py`	Che vùng T1/T2/T3, mặt nạ phổi heuristic, tóm tắt Δ_{occ}.	ảnh + điểm → JSON

Bảng 2. Các script trong `code/scripts/` (cờ bắt buộc lấy từ `argparse`).

Script	Bước	Cờ bắt buộc → đầu ra
`prepare_vinbig.py`	E0	`--train-csv --out-root` + (`--png-dir --meta-csv`) hoặc `--dicom-dir` → `manifest_vinbig.csv`, `vinbig/*.png`
`prepare_nih.py`	E0	`--raw-dir --out-root` → `manifest_nih.csv`, `nih/*.png`
`prepare_jsrt.py`	E0	`--raw-dir --out-root` → `manifest_jsrt.csv`, `jsrt/*.png`
`draw_boxes.py`	E0	`--manifest --data-root --out` → ảnh PNG có vẽ box để kiểm tra bằng mắt
`data_stats.py`	E0	`--manifests --out` → `e0_data.md` và `e0_data_edges.json` (cùng thư mục)
`train_densenet.py`	E1	`--manifests --data-root --out` → dòng JSONL config `B1`
`train_yolo.py`	E1	`--manifests --data-root --work --out` → `best.pt`, dòng JSONL config `B2`
`predict_vlm.py`	E2–E4	`--model --manifests --splits --data-root --config --out` → dòng JSONL (chạy tiếp được nếu bị ngắt)
`train_lora.py`	E3–E4	`--model --mode --manifest --data-root --out` → `final/`, `checkpoint-*`, `timing.json`
`make_report.py`	sau E1–E4	`--manifests --preds --edges-json --out` → 8 file trong `outputs/report/`
`run_audit.py`	E5	`--scorer --manifest --split --data-root --threshold --edges-json --out` → `.json` + `.csv`

4.4 Luồng file



Hình 21. Các file sinh ra theo thứ tự. Tên file `audit/*.json` do bạn đặt qua `--out`; ở đây là ví dụ trong RUNBOOK.

Một dòng manifest

Manifest có 12 cột: `image_id, dataset, path, orig_w, orig_h, label, boxes, split, subtlety, size_mm, patient_id, n_rads`. `path` là đường dẫn tương đối so với `--data-root`. `boxes` là JSON, tọa độ pixel trên ảnh 896 px. `split` là một trong `train, val, test, test_nih, test_jsrt`. `subtlety` và `size_mm` chỉ có ở JSRT.

Một dòng dự đoán JSONL

Mỗi ảnh một dòng. Ví dụ một dòng của VLM (theo `vlm/predict.py`; đã xuống dòng cho dễ đọc):

```
{"image_id": "JPCLN001", "dataset": "jsrt", "split": "test_jsrt",
 "score": 0.734,
 "label_pred": 1,
 "boxes_pred": [[412.0, 300.0, 470.0, 355.0, 0.734]],
 "parse_ok": true,
 "raw_text": "{\"nodule\": \"yes\", \"boxes\": [[412, 300, 470, 355]]}",
 "config": "C3", "seed": 0, "run": 0, "model": "medgemma"}
```

- `score`: $P(\text{yes})$ từ log-prob, dùng cho ROC/AUC và ngưỡng.
- `label_pred`: nhãn đọc được từ câu trả lời sinh ra (1, 0, hoặc `null` nếu không đọc được).
- `boxes_pred`: mỗi box có 5 số; số thứ 5 là điểm. Với VLM, điểm box = `score` của ảnh.
- `parse_ok`: câu trả lời có phải JSON hợp lệ không. `raw_text`: nguyên văn câu trả lời.
- Dòng của B1/B2 (`train_densenet.py`, `train_yolo.py`) có cùng các khóa, trừ `model`; `label_pred` là `null`, `raw_text` rỗng.

> `make_report.py` **đọc lại** `raw_text` bằng parser hiện hành (`reparse_predictions`). Nếu sau này bạn sửa parser, không cần chạy lại mô hình: chỉ cần chạy lại `make_report.py`. Dòng trùng (`config, seed, run, split, image_id`) chỉ giữ dòng cuối. Dòng có `config` là `pilot` bị bỏ qua.

5 Các cấu hình và thí nghiệm

5.1 Bảy cấu hình

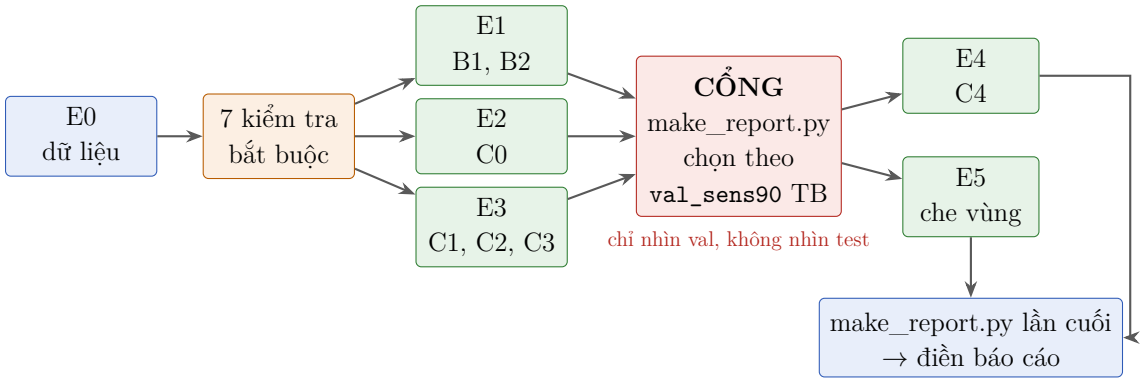
Mã	Cấu hình (spec §5.2)	Đầu ra	Trong code
C0	VLM zero-shot (MedGemma-4B, Qwen2.5-VL-3B), không huấn luyện	JSON	`predict_vlm.py`, không `--adapter`; `--config C0-medgemma/C0-qwen`
C1	LoRA MedGemma-4B, chỉ phân loại	yes/no	`train_lora.py --mode cls`
C2	LoRA MedGemma-4B, phân loại + định vị	yes/no + box	`--mode loc`
C3	C2 + cân bằng lớp (oversampling 1:1, trọng số loss cố định $w_+ = 2{,}0$, không tinh chỉnh)	yes/no + box	`--mode loc --balance --pos-weight 2.0`
C4	Cấu hình tốt nhất trong C1–C3, áp lên Qwen2.5-VL-3B	như nguồn	`train_lora.py --model qwen` + cờ của cấu hình nguồn
B1	DenseNet-121 (bộ phân loại)	xác suất	`train_densenet.py`
B2	YOLOv8 (bộ phát hiện)	box + điểm	`train_yolo.py` (mặc định `yolov8s.pt`)

Mở rộng, không bắt buộc: C5 (VLM nhận box đề xuất từ B2) và C6 (chia ô ảnh). Chỉ làm khi C0–C4, B1, B2 và E5 đã xong.

Siêu tham số	Giá trị mặc định trong code
LoRA (`train/lora.py`)	$r = 16$, $\alpha = 32$, dropout 0,05; chỉ q/k/v/o/gate/up/down_proj của LLM
`train_lora.py`	batch 1, `--grad-accum 8`, `--lr 1e-4`, cosine, warmup 3%, `--epochs 3`, `--neg-per-pos 4.0`, lưu checkpoint mỗi epoch
`train_densenet.py`	ảnh 224 px, batch 32, Adam lr 1e-4, `--epochs 10`, BCE có `pos_weight` = âm/dương; giữ epoch có val AUC cao nhất
`train_yolo.py`	`--weights yolov8s.pt`, imgsz 896, batch 16, `--epochs 50`, `--neg-per-pos 4.0`
`predict_vlm.py`	`--max-new-tokens 96` (C0 dùng 256), `--temperature 0.0`, `--prompt loc`, `--generate all`

5.2 Sáu thí nghiệm E0–E5

Mã	Nội dung	Kết quả cần có	Số lượt
E0	Chuẩn bị dữ liệu, thống kê, hợp nhất box	Bảng mô tả dữ liệu	1
E1	Baseline B1, B2 trên VinBigData; đánh giá trên 3 tập test	AUC, sens@spec0,90, FROC	3 seed mỗi loại
E2	C0 cho 2 VLM; 3 lần (nhiệt độ 0 một lần, 0,7 hai lần)	Bảng zero-shot, tỷ lệ parse lỗi, độ ổn định	3 run × 2 model
E3	LoRA C1, C2, C3 trên MedGemma	Bảng so sánh, đường học	3 cấu hình × 3 seed
E4	C4 trên Qwen2.5-VL-3B	Y khoa vs đa năng	1–3 seed
E5	Che vùng cho C0-medgemma, LoRA tốt nhất, B2; trên `test` và `test_jsrt`	Bảng đổi đáp án, sens theo tầng	3 mô hình × 2 tập

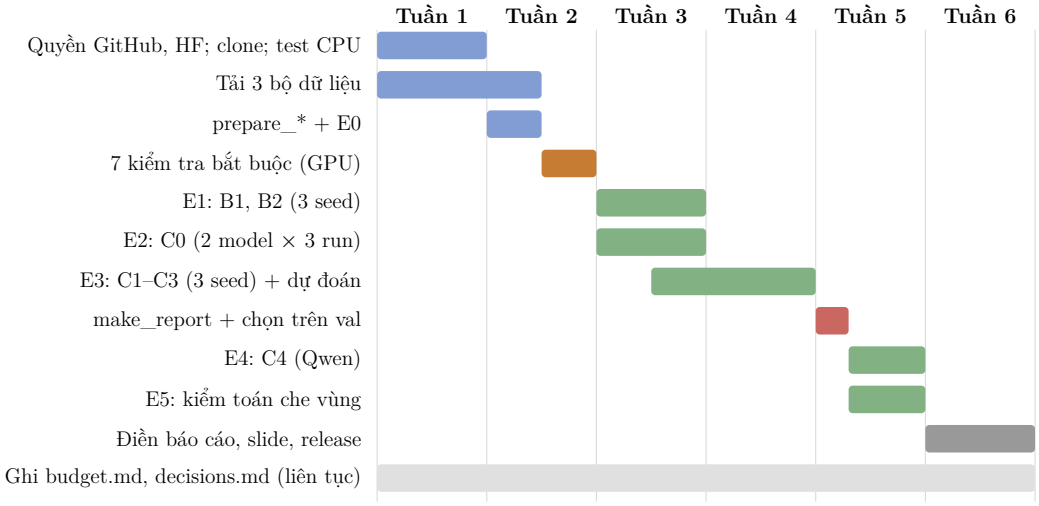


Hình 22. Thứ tự thí nghiệm. Cổng đỏ: chọn "cấu hình tốt nhất" cho E4 và E5 chỉ bằng `val_selection.csv`. Tập test chỉ dùng để chấm, một lần.

Quy tắc "chọn trên val, chấm test một lần". Ngưỡng t_{90}, epoch, cấu hình tốt nhất cho C4 và E5: tất cả chọn trên validation. Các cột của tập test trong `summary.csv` **không được** dùng để chọn. Nếu đã lỡ nhìn test rồi đổi cấu hình, phải ghi rõ vào `docs/decisions.md` và nêu trong báo cáo.

5.3 Lịch đề xuất

Đây là **đề xuất** của người bàn giao, không phải lịch của môn học. Nhóm hãy chỉnh theo hạn nộp thật và ngân sách GPU thật (đo ở kiểm tra mục 7 và pilot E3).



Hình 23. Lịch 6 tuần **đề xuất**. Cam: cổng kiểm tra GPU. Đỏ: cổng chọn cấu hình trên val.

6 Runbook từng bước

> **Nguồn lệnh.** Các khối lệnh đánh dấu [**RUNBOOK**] được chép nguyên từ `code/RUNBOOK.md` (chỉ bỏ hoặc dịch sang tiếng Anh không dấu các chú thích tiếng Việt, vì listings không in được chữ có dấu). Khối đánh dấu [**DỰNG TỪ ARGPARSE**] do sổ tay ghép từ các cờ có thật trong `code/scripts/*.py` theo đúng quy tắc RUNBOOK. Khối đánh dấu [**GỢI Ý**] là lệnh ngoài pipeline (tải dữ liệu, git…), bạn nên kiểm tra lại. Nếu RUNBOOK được cập nhật, **RUNBOOK thắng**.

6.1 Môi trường trên máy cá nhân (CPU)

Dùng để chạy kiểm thử và sửa code. Không cần GPU. Cần Python $\geq$ 3.9 (`pyproject.toml`).

[**GỢI Ý**] (macOS/Linux; Windows dùng `.venv\Scripts\activate`):

```
cd uit-cs315-lung-nodule-cxr-vlm/code
python3 -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
pytest -q
```

Kết quả mong đợi: dòng cuối có `79 passed` (khi đã có torch, xem hộp dưới). Các nhóm phụ thuộc trong `pyproject.toml`: `dev` = pytest; `torchcpu` = torch (CPU); `gpu` = torch, transformers, peft, accelerate, torchxrayvision, ultralytics. Kiểm thử CPU không cần nhóm `gpu`.

> `tests/test_train.py` và một test trong `tests/test_vlm.py` dùng `pytest.importorskip("torch")`: nếu máy chưa có `torch`, các test đó bị *bỏ qua* (skipped) thay vì chạy. Để thấy đủ 79 passed, cài thêm torch bản CPU: `pip install -e ".[dev,torchcpu]"` rồi chạy lại `pytest -q`.

6.2 Clone repo vào Google Drive

Colab đọc code và dữ liệu từ Google Drive. Repo là **public**, nên clone không cần token. Chỉ khi muốn *đẩy* (push) nhánh lên repo gốc, bạn mới cần được mời làm collaborator và xác thực GitHub

(**Personal Access Token** (*PAT*) hoặc `gh auth login`). Cách khác là fork repo rồi mở PR.

[RUNBOOK] Lần đầu tiên (chạy một lần duy nhất, trong một ô Colab): clone repo vào `/content/drive/MyDrive`.

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive
!git clone https://github.com/minhdang900/uit-cs315-lung-nodule-cxr-vlm.git
```

Kết quả: bản clone nằm ở `/content/drive/MyDrive/uit-cs315-lung-nodule-cxr-vlm`. Mỗi phiên sau đó đặt `REPO` bằng đường dẫn này (§6.7). Nếu bạn clone vào chỗ khác, chỉ cần đổi giá trị `REPO`.

> **Xác thực khi push [GỢI Ý].** `git push` sẽ hỏi tên người dùng và mật khẩu; mật khẩu là PAT, không phải mật khẩu GitHub. Nếu ô Colab không cho nhập, đặt PAT vào Colab Secrets (ví dụ tên `GH_TOKEN`) và push bằng `https://{tok}@github.com/...` với `tok = userdata.get("GH_TOKEN")`. Không lưu URL có token vào `.git/config`. Không bao giờ dán token thẳng vào notebook. Dễ nhất là push từ máy cá nhân.

Cách khác [GỢI Ý]: trên máy có Google Drive for Desktop, clone vào thư mục `My Drive` đã đồng bộ; kết quả giống hệt.

Thư mục dữ liệu. Trong bản clone có `data/README.md`. Dữ liệu nằm ở `$REPO/data/raw/` (tải về) và `$REPO/data/processed/` (do `prepare_*` tạo). Cả hai bị gitignore; chỉ `data/README.md` vào git.

> **E0 đã xong (2026-11-13).** Chủ repo đã chuẩn bị sẵn `data/processed/` (4,9 GB). Xin GVHD chia sẻ thư mục dữ liệu trên Google Drive, rồi trong Drive chọn *Sắp xếp* → *Thêm lối tắt* vào `My Drive`, đặt tên lối tắt là `cs315-data` (bên trong có `processed/`). Khi đó biến `DATA` ở §6.7 là `/content/drive/MyDrive/cs315-data`. Không được chia sẻ thì tự tạo lại theo §6.5 và §6.6, rồi đặt `DATA = f"{REPO}/data"`. Danh sách việc còn lại và trạng thái: đầu file `code/RUNBOOK.md`, mục "Trạng thái và việc cần làm".

6.3 Quyền tải MedGemma và HF token

1. Tạo tài khoản https://huggingface.co.
2. Mở trang `google/medgemma-4b-it`. Đọc và chấp nhận điều khoản **Health AI Developer Foundations (HAI-DEF)**. Chờ được cấp quyền (thường ngay lập tức, có thể lâu hơn).
3. Vào *Settings* → *Access Tokens*, tạo token (quyền *Read* là đủ để tải model).
4. Trong Colab: biểu tượng chìa khóa (*Secrets*) ở thanh bên trái → *Add new secret*, tên `HF_TOKEN`, dán token, bật *Notebook access*.

Qwen2.5-VL-3B-Instruct không cần chấp nhận điều khoản riêng, nhưng vẫn tải qua cùng token.

> Không bao giờ dán token vào notebook, code, hay commit. Nếu lỡ lộ token: vào Hugging Face thu hồi (*revoke*) ngay và tạo token mới.

6.4 Colab và GPU

- Cần **Colab Pro** (hoặc gói trả phí tương đương) để chọn GPU **L4** hoặc **A100**. *Runtime* → *Change runtime type* → *GPU*.

- **Không dùng T4.** MedGemma cần bf16 thật (compute capability ≥ 8). Trên T4 (7.5), `load_base` ném lỗi `RuntimeError: MedGemma needs a bf16-capable GPU (L4/A100); fp16 overflows`.
- Colab tính phí theo "compute unit". Ngân sách thật phải đo bằng kiểm tra mục 7 và pilot E3, rồi ghi vào `outputs/budget.md`.

Chưa kiểm chứng được

Người bàn giao chưa chạy GPU lần nào, nên **chưa có số giờ GPU hay số compute unit thực tế**. Spec §7 chỉ đặt quy tắc: nếu một lượt MedGemma vượt 3 giờ thì cắt giảm theo thứ tự (xem §6.10).

6.5 Tải dữ liệu

Giấy phép dữ liệu. Cả ba bộ chỉ được dùng cho nghiên cứu/học thuật theo điều khoản riêng của từng bộ. **Không phân phối lại ảnh** (không commit, không đưa lên Drive công khai, không gửi cho người ngoài nhóm). Phải trích dẫn nguồn trong báo cáo. Hãy đọc nguyên văn điều khoản trên trang của từng bộ; sổ tay này không thay thế điều khoản. Spec "Việc phải làm đầu tiên" mục 1 yêu cầu xác nhận điều khoản trước khi dùng.

Bố cục dữ liệu (theo `data/README.md`): tải dữ liệu thô vào `data/raw/` (gợi ý mỗi bộ một thư mục con: `data/raw/vinbig/`, `data/raw/nih/`, `data/raw/jsrt/`); `prepare_*` ghi kết quả vào `data/processed/`. Cả hai bị gitignore. Nếu bộ thô quá lớn so với dung lượng Drive (ví dụ VinBigData DICOM), có thể tải vào ổ tạm `/content/raw/` của Colab (mất khi hết phiên) và chỉ để `data/processed/` trên Drive.

6.5.1 VinBigData (huấn luyện, validation, test trong phân phối)

- Nguồn: cuộc thi Kaggle `vinbigdata-chest-xray-abnormalities-detection`. Phải bấm chấp nhận luật cuộc thi (*rules*) trước khi tải.
- Cần file `train.csv` (một dòng một box của một bác sĩ: `image_id, class_name, rad_id, x_min, y_min, x_max, y_max`; code chấp nhận cả `rad_ID`) và ảnh train.
- Ảnh gốc là DICOM, tên `<image_id>.dicom`. Bộ gốc rất lớn (cỡ hàng trăm GB, kiểm tra trên trang Kaggle).
- **Bản PNG đã thu nhỏ được hỗ trợ**: `prepare_vinbig.py` có `--png-dir` và `--meta-csv`. Bản PNG phải đi kèm một CSV chứa `image_id` và **kích thước gốc** của ảnh (để quy đổi tọa độ box trong `train.csv`). Mặc định cột chiều cao là `dim0`, chiều rộng là `dim1` (đổi bằng `--h-col`, `--w-col`). Ảnh PNG phải tên `<image_id>.png`.

Đã kiểm chứng (2026-11-13), cách đã dùng: đăng nhập Kaggle bằng `kaggle auth login` (OAuth, không cần lưu token), chấp nhận luật cuộc thi, rồi:

```
cd ../data/raw/vinbig
kaggle competitions download vinbigdata-chest-xray-abnormalities-detection -f train.csv && unzip
    train.csv.zip
kaggle datasets download xhlulu/vinbigdata-chest-xray-resized-png-1024x1024    # 7.35 GB
unzip -q vinbigdata-chest-xray-resized-png-1024x1024.zip 'train/*' 'train_meta.csv'
```

Trong `train_meta.csv`, `dim0`/`dim1` là kích thước *gốc* (823–3408 px; không box nào vượt), nên

> dùng được với cờ mặc định. Ảnh PNG 1024×1024 (đã méo tỷ lệ), code quy về 896×896 theo kích thước gốc. Kết quả E0: xem `docs/decisions.md` D29.

Tùy chọn: tập test chính thức VinDr-CXR. `prepare_vinbig.py` có `--official-test-csv` và `--official-test-dicom`. Khi dùng, VinBigData Kaggle được chia 85/15 (train/val) và tập test chính thức trở thành `test`. Phần này đọc DICOM bắt buộc. Spec §6 ưu tiên dùng tập này nếu truy cập được (3.000 ảnh, 176 ảnh Nodule/Mass).

Chưa kiểm chứng được

Tập test VinDr-CXR nằm trên PhysioNet (cần tài khoản được xác thực). Sổ tay chưa kiểm chứng file nhãn test có cùng cột với `train.csv` (đặc biệt cột `rad_id`) hay không. Code gom box theo `rad_id`; hãy kiểm tra trước khi chạy.

6.5.2 NIH ChestX-ray14 (test ngoài phân phối 1)

- Nguồn: NIH Clinical Center, https://nihcc.app.box.com/v/ChestXray-NIHCC (12 gói `images_001..012.tar.gz`, tổng ≈45 GB). Các link `nihcc.box.com/shared/static/...` cũ (trong script `batch_download_zips.py`) đã hỏng; tải theo `file_id` như trong `scripts/download_nih.sh`.
- Cách nhanh nhất: `bash scripts/download_nih.sh ../data/raw/nih`. Script tải 2 file CSV, tạo symlink `Data_Entry_2017.csv` → `Data_Entry_2017_v2020.csv`, tính trước danh sách ảnh manifest cần (656 ảnh: 164 dương, 492 âm với `--neg-ratio 3`, `seed=0`), rồi đọc lần lượt từng gói và chỉ giữ các ảnh đó. Vẫn phải đọc đủ 45 GB, nhưng chỉ lưu khoảng 0,3 GB. Đổi `--neg-ratio` thì phải chạy lại script.
- `prepare_nih.py --raw-dir` cần trong thư mục đó (code tìm đệ quy):
  - `Data_Entry_2017.csv` (cột dùng: `Image Index`, `Finding Labels`, `Patient ID`);
  - `BBox_List_2017.csv` (cột: `Image Index`, `Finding Label`, `Bbox [x, y, w, h]`);
  - ảnh `*.png` (1024×1024), có thể nằm trong các thư mục con.
- **Phải tải toàn bộ ảnh**, không chỉ ảnh có box: ảnh âm được rút ngẫu nhiên (`seed=0`) từ *mọi* ảnh "No Finding" của bệnh nhân không có ảnh dương, số lượng gấp `--neg-ratio` (mặc định 3) lần số ảnh dương. Thiếu ảnh sẽ gây `KeyError`.

6.5.3 JSRT (test ngoài phân phối 2, có độ mờ)

- Nguồn ảnh (chính thức, miễn phí): http://imgcom.jsrt.or.jp/download/, bản DICOM: `Nodule154images.zip` (806 MB) và `NonNodule93images.zip` (484 MB). Trang cũ `db.jsrt.or.jp` đã ngừng hoạt động.
- File thông tin lâm sàng (tọa độ, kích thước, độ mờ) chính thức chỉ dành cho người đăng ký, mà trang đăng ký đã chết. Đồ án dùng bản sao `CLNDAT_EN.txt` và `CNNDAT_EN.TXT` trong repo HybridGNet (https://github.com/ngaggion/HybridGNet); chỉ dùng cho nghiên cứu, không commit (quyết định D28).
- Cách nhanh nhất: `bash scripts/download_jsrt.sh ../data/raw/jsrt` (tải ảnh, giải nén, tải 2 file lâm sàng).
- 247 ảnh: 154 có nốt (`JPCLN*`), 93 không có nốt (`JPCNN*`); 2048×2048, 0,175 mm/pixel, MONOCHROME2.

- `prepare_jsrt.py` nhận cả bản `.IMG` cũ lẫn bản DICOM mới (ưu tiên `.IMG` nếu có), và nhận tên file lâm sàng `CLNDAT_EN.txt`/`CLNDAT_E.TXT`, `NNDAT_EN.txt`/`CNNDAT_EN.TXT`/`CNNDAT_E.TXT` (không phân biệt hoa thường, tìm đệ quy).

> **Đã kiểm chứng (2026-11-13):** chạy `prepare_jsrt.py` trên bản DICOM cho 154 ảnh dương, 93 ảnh âm; `draw_boxes.py` cho thấy box nằm đúng nốt (ví dụ JPCLN001 thùy trên trái, JPCLN004 thùy trên phải), ảnh không bị đảo màu.

6.5.4 Lệnh tải gợi ý

[**GỢI Ý**] Kaggle CLI cần file API token `kaggle.json` (Kaggle → Settings → API). Không commit file này.

```
pip install -q kaggle
mkdir -p ~/.kaggle && cp kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
kaggle competitions download -c vinbigdata-chest-xray-abnormalities-detection -p ../data/raw/vinbig
```

6.6 Chuẩn bị dữ liệu (phần đầu E0)

Chạy từ `code/`, với `M=../data/processed`. Các script này chỉ cần CPU (có thể chạy trên Colab CPU hoặc máy cá nhân có đủ dữ liệu). RUNBOOK chỉ liệt kê cờ bắt buộc; dưới đây là lệnh đầy đủ [**DỰNG TỪ ARGPARSE**], đường dẫn `../data/raw/...` theo bố cục ở trên (đổi nếu bạn để dữ liệu chỗ khác):

```
%%bash
export M=../data/processed
# VinBigData, PNG variant (needs original sizes in --meta-csv)
python scripts/prepare_vinbig.py --train-csv ../data/raw/vinbig/train.csv --png-dir ../data/raw/
    vinbig/train --meta-csv ../data/raw/vinbig/train_meta.csv --out-root $M
# ... OR VinBigData, DICOM (sizes are read from the DICOM headers)
# python scripts/prepare_vinbig.py --train-csv ../data/raw/vinbig/train.csv --dicom-dir ../data/raw/
    vinbig/train --out-root $M
python scripts/prepare_nih.py --raw-dir ../data/raw/nih --out-root $M
python scripts/prepare_jsrt.py --raw-dir ../data/raw/jsrt --out-root $M
```

Kết quả mong đợi

- Trong `data/processed/`: `manifest_vinbig.csv`, `manifest_nih.csv`, `manifest_jsrt.csv` và các thư mục `vinbig/`, `nih/`, `jsrt/` chứa PNG 896×896 xám.
- `prepare_vinbig.py` in bảng `count`/`sum` theo split: tỷ lệ `train:val:test` xấp xỉ 70:10:20, tỷ lệ dương trong mỗi split gần bằng nhau (chia phân tầng).
- `prepare_nih.py` in số ảnh nhãn 1 và nhãn 0; nhãn 0 gấp 3 lần nhãn 1 (nếu đủ ảnh âm).
- `prepare_jsrt.py` in `1: 154`, `0: 93`.

Chưa kiểm chứng được

Spec ghi 410/15.000 ảnh Nodule/Mass (theo bài báo VinDr-CXR). Code coi một ảnh là dương nếu *ít nhất một* bác sĩ khoanh Nodule/Mass, và coi *mọi* ảnh còn lại là âm (kể cả ảnh có bệnh khác, tức "âm khó"). Vì vậy số ca dương thật có thể khác 410. Ghi con số thật vào E0 và báo cáo.

6.7 Mỗi phiên Colab (RUNBOOK §0)

[**RUNBOOK**] Ô Python đầu tiên của mỗi phiên. Đổi `REPO` nếu bạn clone vào chỗ khác (§6.2):

```
from google.colab import drive, userdata
drive.mount('/content/drive')
REPO = "/content/drive/MyDrive/uit-cs315-lung-nodule-cxr-vlm"
DATA = "/content/drive/MyDrive/cs315-data"   # shared folder shortcut; or f"{REPO}/data"
%cd {REPO}/code
import os; os.environ["HF_TOKEN"] = userdata.get("HF_TOKEN")   # from Colab Secrets
!pip install -q -e ".[gpu]"
!huggingface-cli login --token $HF_TOKEN
!mkdir -p /content/data ../outputs && rsync -a "{DATA}/processed/" /content/data/
```

Giải thích từng dòng. Gắn Drive; đặt `REPO` (bản clone) và `DATA` (thư mục dữ liệu được chia sẻ, xem §6.2); chuyển vào `$REPO/code`; nạp token vào biến môi trường; cài gói cùng nhóm `gpu`; đăng nhập Hugging Face; chép ảnh *và* manifest từ Drive sang ổ local `/content/data` (đọc từ Drive rất chậm).

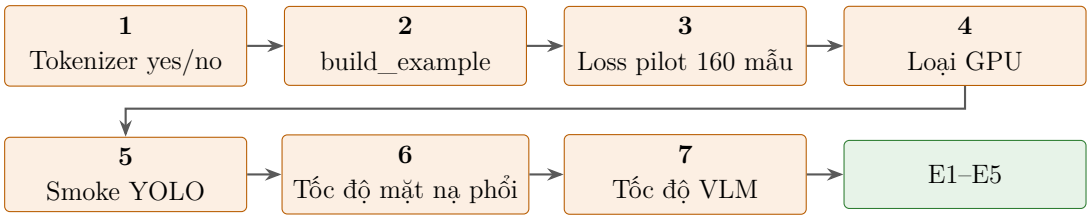
Biến shell không giữ qua các dòng `!`. Vì vậy mọi ô chạy lệnh phải là ô `%%bash` và bắt đầu bằng [**RUNBOOK**]:

```
%%bash
export M=/content/data D=/content/data O=../outputs
```

`M`: manifest và `D`: gốc ảnh (cả hai ở ổ local sau `rsync`). `O`: kết quả (trong bản clone trên Drive, gitignore). Các khối lệnh bash bên dưới đều là nội dung đặt *sau* dòng `export` này.

6.8 Bảy kiểm tra bắt buộc (phiên GPU đầu tiên)

> Làm lần lượt, **trước** E1–E5. Mục nào không đạt thì **dừng và sửa**. Code GPU mới chỉ được kiểm tra cú pháp; đây là lần đầu nó chạy thật. Ghi kết quả từng mục (đạt/không, con số đo được) vào `outputs/budget.md` hoặc một issue trên GitHub.



Hình 24. Bảy kiểm tra trước khi chạy đủ. Gợi ý: làm mục 4 đầu tiên, vì nếu GPU sai thì mọi mục khác đều vô nghĩa.

Mục 1 – Tokenizer cho điểm yes/no

[**RUNBOOK**] Chạy cho `medgemma`, rồi lặp lại cho `qwen`:

```
from nodulecxr.vlm.hf import load_base
from nodulecxr.vlm.chat import build_chat_text
from nodulecxr.prompts import QUESTION_LOC, ANSWER_PREFIX, build_target
_, processor = load_base("medgemma")   # repeat with "qwen"
tok = processor.tokenizer
base = build_chat_text(processor, QUESTION_LOC) + ANSWER_PREFIX
for w in ("yes", "no"):
    print(w, tok.convert_ids_to_tokens(tok(base + w, add_special_tokens=False)["input_ids"])[-8:])
```

```
tgt = build_chat_text(processor, QUESTION_LOC, build_target(1, [[1, 2, 3, 4]]))
print("target", tok.convert_ids_to_tokens(tok(tgt, add_special_tokens=False)["input_ids"]))
```

Đạt khi: (a) sau điểm phân kỳ giữa `yes` và `no`, token chỉ là chính từ trả lời, không gộp dấu nháy (`"yes` hay `yes"` là **sai**); (b) trong target huấn luyện, ngay sau `ANSWER_PREFIX` là đúng token `yes`. **Vì sao**: nếu dấu nháy bị gộp vào token, điểm $P(\text{yes})$ so sai hai chuỗi và mọi AUC đều sai.

Mục 2 – Một lần `build_example` trên ảnh thật

[**RUNBOOK**] (dùng tiếp `processor`, `QUESTION_LOC`, `build_target` từ mục 1):

```
from nodulecxr.records import read_manifest
from nodulecxr.images import load_image
from nodulecxr.train.sft_data import build_example
df = read_manifest("../data/processed/manifest_vinbig.csv"); r = df[df.label == 1].iloc[0]
ex = build_example(processor, load_image("/content/data", r.path), QUESTION_LOC, build_target(1, [[1,
     2, 3, 4]]))
print({k: v.shape for k, v in ex.items()})
```

Đạt khi: in ra các shape, không có `ValueError: chat template: prompt is not a prefix of prompt+answer`. Làm cho cả hai model.

Mục 3 – Loss đầu tiên của pilot 160 mẫu

RUNBOOK: thêm `--max-train 160 --epochs 1` vào một lệnh `train_lora.py` của E3. Ví dụ [**DỰNG TỪ ARGPARSE**] (thư mục `--out` tùy bạn, miễn không trùng `C1/C2/C3`):

```
python scripts/train_lora.py --model medgemma --mode loc --manifest $M/manifest_vinbig.csv --data-
    root $D --seed 0 --max-train 160 --epochs 1 --out $O/lora/pilot
```

160 mẫu, grad-accum 8 → 20 bước tối ưu; `loss` được ghi ở bước 10 và 20 (`logging_steps=10`). **Đạt khi**: `loss` dòng đầu khoảng **0,5–3**. Nếu khoảng **10–25**: gradient accumulation bị chia hai lần — dừng và sửa. Đọc thêm `train_loss` cuối và `timing.json` (giây huấn luyện) để ước tính thời gian một lượt đầy đủ.

Mục 4 – Loại GPU

[**RUNBOOK**]

```
import torch; print(torch.cuda.get_device_name(), torch.cuda.get_device_capability())
```

Đạt khi: L4 hoặc A100, số đầu của capability ≥ 8. Đừng tin `torch.cuda.is_bf16_supported()`: T4 (7.5) cũng trả `True` nhờ giả lập.

Mục 5 – Smoke YOLO 1 epoch

[**RUNBOOK**] Cách 1 (`<work>` là thư mục đã có `yolo_data_seed0/data.yaml`, ví dụ do cách 2 tạo):

```
from ultralytics import YOLO
m = YOLO("yolov8s.pt"); m.train(data="<work>/yolo_data_seed0/data.yaml", imgsz=896, epochs=1, project
    ="/content/yolo_smoke", name="s", exist_ok=True)
print(m.trainer.best)   # file must exist
```

[**RUNBOOK**] Cách 2, dùng đủ cờ bắt buộc của `train_yolo.py`:

```
%%bash
```

```
export M=/content/data D=/content/data O=../outputs
python scripts/train_yolo.py --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/manifest_jsrt.
    csv --data-root $D --epochs 1 --work /content/yolo_smoke --out /content/smoke.jsonl
ls /content/yolo_smoke/seed0/weights/best.pt
```

Cách 2 sau khi train còn dự đoán toàn bộ val, test, test_nih, test_jsrt nên chậm; có thể dừng ô khi `best.pt` đã xuất hiện.

Mục 6 – Thời gian mặt nạ phổi

[**RUNBOOK**] (PSPNet của torchxrayvision chạy trên CPU; nhãn in đã đổi sang ASCII):

```
import sys, time; sys.path.insert(0, "scripts")
from run_audit import lung_mask_xrv
from nodulecxr.records import read_manifest
from nodulecxr.images import load_image
df = read_manifest("../data/processed/manifest_jsrt.csv").head(10)
lung_mask_xrv(load_image("/content/data", df.iloc[0].path))   # warm-up, not timed
t = time.time()
for r in df.itertuples(): lung_mask_xrv(load_image("/content/data", r.path))
print((time.time() - t) / len(df), "s/img")
```

Nếu quá chậm: dùng `--lung-mask heuristic` ở E5 và ghi quyết định vào `$O/budget.md`.

Mục 7 – Thời gian suy luận mỗi ảnh của VLM

[**RUNBOOK**] 50 ảnh val mỗi model, config `pilot`, ghi ra ngoài `$O/preds/`:

```
for m in medgemma qwen; do
  python scripts/predict_vlm.py --model $m --manifests $M/manifest_vinbig.csv --splits val --limit 50
      --data-root $D --config pilot --max-new-tokens 256 --out /content/pilot_$m.jsonl
done
```

Đọc `s/it` ở thanh tqdm (mỗi `it` = một ảnh, gồm cả tính điểm và sinh văn bản). Ghi vào `$O/budget.md`. Chi phí C0 ≈ giây/ảnh × số ảnh của `val test test_nih test_jsrt` × 3 run × 2 model. Nhân tiện, mở `/content/pilot_medgemma.jsonl` xem `raw_text` và `parse_ok`: MedGemma có xuất JSON và box không? (spec "Việc phải làm đầu tiên" mục 3).

6.9 Chạy các thí nghiệm

6.9.1 E0 – Dữ liệu

Sau 3 script `prepare_*` (§6.6), [**RUNBOOK**]:

```
for d in vinbig nih jsrt; do python scripts/draw_boxes.py --manifest $M/manifest_$d.csv --data-root
    $M --out $O/qa/$d -n 8; done
python scripts/data_stats.py --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/manifest_jsrt.
    csv --out $O/e0_data.md
```

Kiểm tra: mở `outputs/qa/{vinbig,nih,jsrt}/` bằng mắt; box đỏ phải nằm đúng trên nốt. Đọc `outputs/e0_data.md`. **Đầu ra**: `$O/e0_data.md`, `$O/e0_data_edges.json` (ngưỡng tertile, dùng ở báo cáo và E5).

6.9.2 E1 – Baseline CNN (3 seed mỗi loại)

[**RUNBOOK**]

```
for s in 0 1 2; do
```

```
  python scripts/train_densenet.py --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/
    manifest_jsrt.csv --data-root $D --seed $s --out $O/preds/B1.jsonl
  python scripts/train_yolo.py --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/
    manifest_jsrt.csv --data-root $D --seed $s --work /content/yolo --out $O/preds/B2.jsonl
  mkdir -p $O/yolo/seed$s/weights && cp /content/yolo/seed$s/weights/best.pt $O/yolo/seed$s/weights/
    best.pt
done
```

Kiểm tra: log DenseNet in `epoch k val AUC ...` mỗi epoch; `B1.jsonl`, `B2.jsonl` có dòng cho mọi ảnh không thuộc train; `$O/yolo/seed{0,1,2}/weights/best.pt` tồn tại trên Drive. Dòng `cp` bắt buộc: `/content` mất khi hết phiên, và E5 cần `best.pt`.

6.9.3 E2 – Zero-shot C0

[**RUNBOOK**]

```
for m in medgemma qwen; do
  python scripts/predict_vlm.py --model $m --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/
    manifest_jsrt.csv --splits val test test_nih test_jsrt --data-root $D --config C0-$m --run 0 --
    temperature 0 --max-new-tokens 256 --out $O/preds/C0-$m.jsonl
  for r in 1 2; do
    python scripts/predict_vlm.py --model $m --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv
    $M/manifest_jsrt.csv --splits val test test_nih test_jsrt --data-root $D --config C0-$m --run $r
    --temperature 0.7 --max-new-tokens 256 --out $O/preds/C0-$m.jsonl
  done
done
```

Script tự bỏ qua ảnh đã có trong file (cùng `config, seed, run`); Colab ngắt thì chạy lại *đúng lệnh cũ*. Bảng chính chỉ dùng run 0 (nhiệt độ 0); run 1–2 dùng cho `run_agreement.csv`.

6.9.4 E3 – LoRA MedGemma C1–C3 (3 seed)

Trước tiên: pilot ở kiểm tra mục 3. Sau đó [**RUNBOOK**] huấn luyện:

```
for s in 0 1 2; do
  python scripts/train_lora.py --model medgemma --mode cls --manifest $M/manifest_vinbig.csv --data-
    root $D --seed $s --out $O/lora/C1-s$s
  python scripts/train_lora.py --model medgemma --mode loc --manifest $M/manifest_vinbig.csv --data-
    root $D --seed $s --out $O/lora/C2-s$s
  python scripts/train_lora.py --model medgemma --mode loc --balance --pos-weight 2.0 --manifest $M/
    manifest_vinbig.csv --data-root $D --seed $s --out $O/lora/C3-s$s
done
```

Dự đoán. [**RUNBOOK**] ví dụ cho C3:

```
for s in 0 1 2; do
  python scripts/predict_vlm.py --model medgemma --adapter $O/lora/C3-s$s/final --prompt loc --
    generate all --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/manifest_jsrt.csv --splits
     val test test_nih test_jsrt --data-root $D --config C3 --seed $s --out $O/preds/C3.jsonl
done
```

C1 và C2 [**DỰNG TỪ ARGPARSE**] theo quy tắc RUNBOOK (C1: `--prompt cls --generate all --max-new-tokens 16`; C2: `--prompt loc --generate all`):

```
for s in 0 1 2; do
  python scripts/predict_vlm.py --model medgemma --adapter $O/lora/C1-s$s/final --prompt cls --
    generate all --max-new-tokens 16 --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/
    manifest_jsrt.csv --splits val test test_nih test_jsrt --data-root $D --config C1 --seed $s --out
```

```
    $O/preds/C1.jsonl
  python scripts/predict_vlm.py --model medgemma --adapter $O/lora/C2-s$s/final --prompt loc --
    generate all --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/manifest_jsrt.csv --splits
     val test test_nih test_jsrt --data-root $D --config C2 --seed $s --out $O/preds/C2.jsonl
done
```

Với C1, `--generate none` chỉ cho điểm và để trống các cột `raw_sens`, `no_on_pos`, `yes_on_neg`, `unparsed`; vì vậy nên dùng `--generate all`.

Đường học (RUNBOOK): chạy `predict_vlm.py --splits val --generate none` cho từng `checkpoint-*` của seed 0, với `--config <cfg>-ep<k>` và cùng `--prompt` của cấu hình nguồn.

> Nếu ghi các dự đoán đường học vào `$O/preds/`, `make_report.py` sẽ đưa các dòng `C3-ep1`, `C3-ep2`… vào `val_selection.csv` (chúng chỉ có split val nên không vào bảng test). Khi chọn cấu hình tốt nhất cho E4/E5, **chỉ so C1, C2, C3**; bỏ qua các dòng `-ep`.

6.9.5 Báo cáo trung gian (cổng chọn cấu hình)

[**RUNBOOK**]

```
python scripts/make_report.py --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/manifest_jsrt.
    csv --preds "$O/preds/*.jsonl" --edges-json $O/e0_data_edges.json --out $O/report
```

Mở `$O/report/val_selection.csv`. Tính trung bình `val_sens90` qua seed cho C1, C2, C3. Cấu hình cao nhất là "LoRA tốt nhất". `val_auc` chỉ để tham khảo. Ghi lựa chọn và con số vào `docs/decisions.md`.

[**GỢI Ý**] đọc nhanh bằng pandas:

```
import pandas as pd
v = pd.read_csv("../outputs/report/val_selection.csv")
print(v[v.config.isin(["C1", "C2", "C3"])].groupby("config").val_sens90.agg(["mean", "std", "count"])
    )
```

6.9.6 E4 – C4 trên Qwen2.5-VL-3B (1–3 seed)

RUNBOOK: chạy lại `train_lora.py --model qwen` với *đúng các cờ* của cấu hình tốt nhất, rồi dự đoán với `--model qwen --config C4` và đúng `--prompt`/`--generate` (cùng `--max-new-tokens`) của cấu hình nguồn. Ví dụ [**DỰNG TỪ ARGPARSE**] nếu cấu hình tốt nhất là C3, seed 0:

```
python scripts/train_lora.py --model qwen --mode loc --balance --pos-weight 2.0 --manifest $M/
    manifest_vinbig.csv --data-root $D --seed 0 --out $O/lora/C4-s0
python scripts/predict_vlm.py --model qwen --adapter $O/lora/C4-s0/final --prompt loc --generate all
    --manifests $M/manifest_vinbig.csv $M/manifest_nih.csv $M/manifest_jsrt.csv --splits val test
    test_nih test_jsrt --data-root $D --config C4 --seed 0 --out $O/preds/C4.jsonl
```

Nếu tốt nhất là C1: thay `--mode loc --balance --pos-weight 2.0` bằng `--mode cls`, và dự đoán với `--prompt cls --generate all --max-new-tokens 16`.

6.9.7 E5 – Kiểm toán che vùng

Cấu hình: C0-medgemma, LoRA tốt nhất (adapter seed 0), B2 (seed 0). Hai tập:

- `test`, dùng `manifest_vinbig.csv`;
- `test_jsrt`, dùng `manifest_jsrt.csv`.

Ngưỡng lấy từ `$O/report/thresholds.json`, khóa `config|seed|run`. [**GỢI Ý**] đọc ngưỡng:

```
python -c "import json; t=json.load(open('../outputs/report/thresholds.json')); print(t['C3|0|0']['
    t90'], t['C0-medgemma|0|0']['t90'], t['B2|0|0']['t90'])"
```

[**RUNBOOK**] (ví dụ LoRA tốt nhất là C3; thay `<t90...>` bằng số vừa đọc):

```
# best LoRA (example: C3, seed 0; if C1, add --prompt cls)
python scripts/run_audit.py --scorer vlm --model medgemma --adapter $O/lora/C3-s0/final --manifest $M
    /manifest_jsrt.csv --split test_jsrt --data-root $D --threshold <t90> --edges-json $O/
    e0_data_edges.json --out $O/audit/C3_jsrt.json
# C0 zero-shot (no --adapter)
python scripts/run_audit.py --scorer vlm --model medgemma --manifest $M/manifest_jsrt.csv --split
    test_jsrt --data-root $D --threshold <t90 of C0-medgemma|0|0> --edges-json $O/e0_data_edges.json
    --out $O/audit/C0_jsrt.json
# B2 YOLO
python scripts/run_audit.py --scorer yolo --yolo-weights $O/yolo/seed0/weights/best.pt --manifest $M/
    manifest_jsrt.csv --split test_jsrt --data-root $D --threshold <t90 of B2|0|0> --edges-json $O/
    e0_data_edges.json --out $O/audit/B2_jsrt.json
```

Lặp lại cho VinBigData test: đổi thành `--manifest $M/manifest_vinbig.csv` và `--split test`, đổi tên `--out` (ví dụ `C3_test.json`). Thêm `--lung-mask heuristic` nếu mặt nạ PSPNet quá chậm (mục 6).

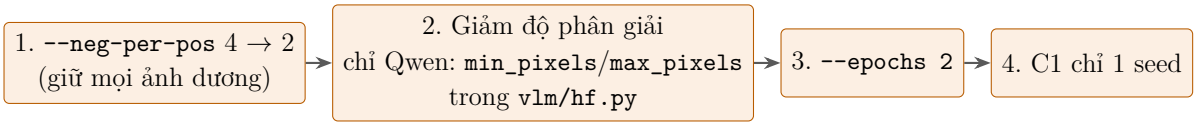
Đọc kết quả: file `.json` có `n`, `n_orig_yes`, `n_t2_ok`, `flip_t1_all`, `flip_t1`, `flip_t2`, `delta`, `yes_rate_t3`, cùng `by_size` và (JSRT) `by_subtlety`. **Số cần báo cáo là `delta`**, kèm `n_t2_ok`. File `.csv` cùng tên chứa điểm từng ảnh.

6.9.8 Báo cáo cuối

Chạy lại `make_report.py` (cùng lệnh ở trên) sau khi có C4. Thư mục `$O/report/` khi đó có đủ 8 file.

6.10 Ngân sách và cắt giảm

Spec §7: lượt LoRA đầu tiên dùng để đo thời gian. **Nếu một lượt vượt 3 giờ**, cắt theo đúng thứ tự (RUNBOOK):



Hình 25. Thứ tự cắt giảm. Bước 2 không áp dụng cho MedGemma (processor cố định 896 px) và không có cờ dòng lệnh.

Ghi *mọi* quyết định vào `outputs/budget.md`: ngày, đo được bao nhiêu, cắt bước nào, áp cho cấu hình nào. `outputs/` không vào git, nên tóm tắt các quyết định cắt giảm cũng phải chép sang `docs/decisions.md`.

6.11 Lỗi thường gặp

Triệu chứng	Nguyên nhân và cách sửa
`RuntimeError: MedGemma needs a bf16-capable GPU (L4/A100)`	Đang ở T4 hoặc CPU. Đổi runtime sang L4/A100. Kiểm tra lại bằng mục 4.

Triệu chứng	Nguyên nhân và cách sửa
`401/403`, `GatedRepoError`, "access to model is restricted"	Chưa được cấp quyền MedGemma, hoặc token sai/thiếu. Kiểm tra: đã chấp nhận HAI-DEF trên đúng tài khoản; secret tên đúng `HF_TOKEN`; đã bật *Notebook access*; đã chạy ô §0.
`SecretNotFoundError` khi `userdata.get`	Chưa tạo secret, hoặc gõ sai tên.
Colab ngắt kết nối giữa chừng	`predict_vlm.py`: chạy lại đúng lệnh, nó chạy tiếp từ ảnh chưa làm. `train_lora.py`, `train_densenet.py`, `train_yolo.py`: **không** tự chạy tiếp; chạy lại sẽ huấn luyện từ đầu. B1/B2 sẽ ghi thêm dòng trùng vào JSONL; `make_report.py` giữ dòng cuối nên vẫn đúng. Nếu một dòng JSONL cuối bị cắt dở, xóa dòng đó.
`CUDA out of memory`	Batch đã là 1 và đã bật gradient checkpointing. Thử: khởi động lại runtime (giải phóng model đã tải ở các ô kiểm tra), dùng A100 thay L4. Với Qwen có thể giảm `max_pixels` trong `vlm/hf.py` (bước 2 của quy tắc ngân sách). Ghi mọi thay đổi vào `budget.md`.
`loss` ban đầu khoảng 10–25	Gradient accumulation bị chia hai lần (mục 3). Dừng, không chạy E3; báo lại trong issue.
`ValueError: chat template: prompt is not a prefix…`	Chat template của model/phiên bản transformers khác giả định. Dừng; ghi phiên bản `transformers`; không tự ý bỏ kiểm tra.
`KeyError` trong `prepare_nih.py`	Thiếu ảnh NIH: phải tải đủ các gói ảnh.
`StopIteration` trong `prepare_jsrt.py`	Thiếu file `.IMG` của một ảnh có trong file TXT.
`FileNotFoundError: ...CLNDAT_EN.txt`	Sai tên hoặc vị trí file thông tin JSRT; phải nằm ngay trong `--raw-dir`.
Box vẽ ở E0 bị lệch	Kích thước gốc sai (`--meta-csv`, `--h-col`, `--w-col`). Sửa rồi xóa `data/processed/vinbig/` và chạy lại (script bỏ qua PNG đã có).
Chạy rất chậm khi đọc ảnh	Đang đọc từ Drive. Dùng `--data-root $D` (`/content/data`) sau lệnh `rsync` ở §0.
`t90` là `Infinity`	Mô hình cho điểm gần như bằng nhau trên mọi ảnh val (chế độ báo bình thường cực đoan). Không sửa; báo cáo trung thực, sensitivity sẽ là 0.
Mất `best.pt` của YOLO	Quên dòng `cp` ở E1. Phải huấn luyện lại seed đó.

6.12 Điền báo cáo từ kết quả

Báo cáo nằm ở `report/`. Các ô [–] màu đỏ chờ số. Biên dịch: `latexmk -xelatex main.tex` trong `report/`.

Bảng 4. Ánh xạ file kết quả → vị trí trong báo cáo.

Vị trí trong báo cáo	File nguồn	Cột / khóa
Bảng `tab:data-stats` (`04a_du_lieu.tex`)	`outputs/e0_data.md`	`n`, `n_pos`, `prevalence` theo split; tầng kích thước
Bảng `tab:main` (`04b_ket_qua.tex`)	`outputs/report/summary.csv`	`sens_mean` ± `sens_sd` (sens ở t_{90}), `auc_roc_mean`; theo `split` = `test`, `test_nih`, `test_jsrt`

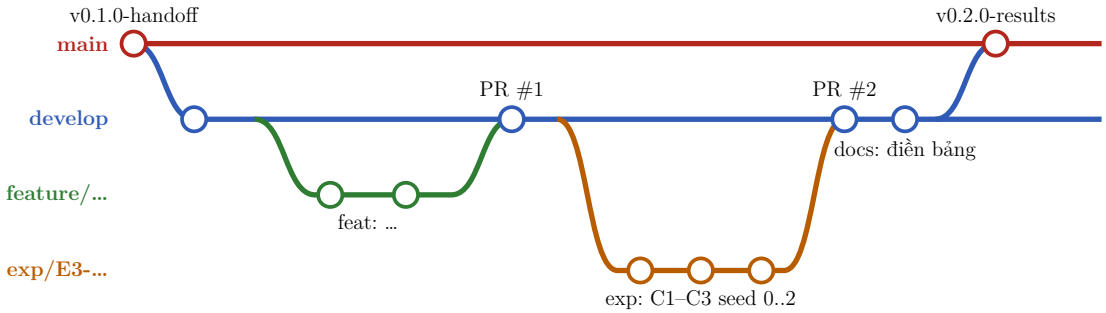
Vị trí trong báo cáo	File nguồn	Cột / khóa
CI 95% kèm bảng chính	`summary_per_seed.csv`	`sens_lo`, `sens_hi`, `auc_roc_lo`, `auc_roc_hi` (theo seed)
Bảng `tab:loc`	`summary.csv`	`mean_best_iou_mean`, `hit_iou30_mean`, `hit_iou50_mean` (`test`); `jsrt_median_dist_mm_mean` (`test_jsrt`). Trừ `hit_iou30`, các cột này không có trong `summary.md`.
Mục "Chế độ báo bình thường"	`summary.csv`	`no_on_pos`, `yes_on_neg`, `unparsed`, `raw_sens`, `ece`, `balanced_brier`; đường học từ dự đoán `-ep<k>`
Độ ổn định C0 (E2)	`run_agreement.csv`	`agreement`, `n_images`
Bảng `tab:audit-res`	`outputs/audit/*.json`	`flip_t1`, `flip_t2`, `delta` (`n_t2_ok`); cột T3 "ảnh âm → no" = 1− `yes_rate_t3`
Biểu đồ sens theo tầng	`strata.csv`; `audit/*.json`	`kind` (`size`/`subtlety`), `stratum`, `sensitivity`, `count`; `by_size`, `by_subtlety`
p-value so sánh cặp	`comparisons.csv`	`p_sens_holm`, `p_spec_holm`, `p_auc_holm` (seed 0, run 0)
Chọn cấu hình (phương pháp)	`val_selection.csv`	`val_sens90` trung bình qua seed
FROC, AUC-PR, PPV	`summary.csv`	`froc_0.5fp_mean`, `froc_1fp_mean`, `auc_pr_mean`, `ppv_prev_1pct_mean`
Tóm tắt, Kết luận (`00_tom_tat.tex`, `05_ket_luan.tex`)	tổng hợp các bảng trên	trả lời Q1, Q2

Cách điền. Mỗi lần điền số, ghi rõ trong commit: số lấy từ file nào, chạy ngày nào (ví dụ `docs: fill tab:main from outputs/report/summary.csv (2026-12-01)`). Báo cáo số làm tròn 3 chữ số thập phân, dùng dấu phẩy thập phân theo văn phong tiếng Việt của báo cáo. Ô nào không có số (ví dụ C1 không có box) ghi "—" và giải thích.

7 Quy trình làm việc nhóm với Git/GitHub

7.1 Mô hình nhánh (git-flow rút gọn)

- `main`: bản ổn định, đã phát hành, có tag. Tag `v0.1.0-handoff` = trạng thái lúc bàn giao.
- `develop`: nhánh tích hợp. Mọi việc gộp vào đây trước.
- Nhánh làm việc, tách từ `develop`:
  - `feature/<tên-ngắn>`: thay đổi code (ví dụ `feature/qwen-max-pixels-flag`).
  - `exp/<E#-tên-ngắn>`: chạy thí nghiệm, chỉnh cấu hình (ví dụ `exp/E3-lora-medgemma`).
  - `docs/<tên>`: báo cáo, tài liệu (ví dụ `docs/fill-tab-main`).
- Gộp: mở **Pull Request** (PR) vào `develop` → review → squash hoặc merge.
- Phát hành: PR `develop` → `main`, rồi gắn tag (ví dụ `v0.2.0-results`).



Hình 26. Mô hình nhánh. Không ai commit thẳng vào `main` hay `develop`.

7.2 Các lệnh từng bước

[**GỢI Ý**] Lần đầu: clone và lấy nhánh `develop`:

```
git clone https://github.com/minhdang900/uit-cs315-lung-nodule-cxr-vlm.git
cd uit-cs315-lung-nodule-cxr-vlm
git checkout develop
git config user.name "Your Name"
git config user.email "you@example.com"
```

Mỗi việc mới:

```
git checkout develop
git pull origin develop                    # always start from the latest develop
git checkout -b feature/qwen-max-pixels-flag
# ... edit code ...
cd code && pytest -q && cd ..              # tests must pass before pushing
git status                                 # check: no data, outputs, weights, tokens
git add code/src/nodulecxr/vlm/hf.py code/tests/test_vlm.py
git commit -m "feat: add max-pixels option for Qwen processor"
git push -u origin feature/qwen-max-pixels-flag
```

Mở PR trên GitHub (*Compare & pull request*), base = `develop`. Hoặc dùng GitHub CLI:

```
gh pr create --base develop --title "feat: add max-pixels option for Qwen" --body "Why, what changed,
    how tested"
```

Review và gộp:

- Ít nhất **một bạn khác** đọc PR. Người review kiểm tra: test qua; không có file cấm; lệnh mới có trong RUNBOOK; quyết định mới có trong `docs/decisions.md`.
- Gộp bằng *Squash and merge* (một commit gọn) hoặc *Merge*. Xóa nhánh sau khi gộp.

Phát hành:

```
gh pr create --base main --head develop --title "release: v0.2.0-results"
# after the PR is merged:
git checkout main && git pull origin main
git tag -a v0.2.0-results -m "E0-E5 results filled in report"
git push origin v0.2.0-results
```

7.3 Quy ước commit

Dùng **conventional commits**: `<loại>: <mô tả ngắn, thì hiện tại>`.

Loại	Dùng khi	Ví dụ
`feat:`	thêm chức năng	`feat: add --official-test support to E0 docs`
`fix:`	sửa lỗi	`fix: parse boxes when model returns bbox_2d`
`docs:`	báo cáo, sổ tay, RUNBOOK	`docs: fill tab:main from summary.csv`
`exp:`	cấu hình/nhật ký thí nghiệm	`exp: E3 budget cut neg-per-pos 2`
`chore:`	việc lặt vặt, cấu hình repo	`chore: update .gitignore`

7.4 Những thứ không bao giờ commit

- **Dữ liệu**: `data/`, ảnh `*.png` của bệnh nhân, `*.dcm`, `*.dicom`, file `.IMG`, file `.zip`. Lý do: giấy phép cấm phân phối lại.
- **Trọng số mô hình**: `*.pt`, `*.safetensors`, `*.ckpt`, thư mục adapter LoRA. Lý do: nặng, và có thể phải tuân điều khoản HAI-DEF.
- **Kết quả chạy**: `outputs/`. Lý do: lớn, sinh lại được; số liệu cần giữ thì chép vào báo cáo.
- **Bí mật**: HF token, GitHub PAT, `kaggle.json`, `.env`.
- **PDF có bản quyền**: bài báo trong `docs/lit-review/**/papers/`, giáo trình PDF ở gốc thư mục môn học.
- **Dữ liệu cá nhân**: file danh sách nhóm `*.xlsx`.

File `.gitignore` ở gốc repo chặn phần lớn các mẫu này: toàn bộ `data/` (trừ `data/README.md`), `outputs/`, `*.pt`, `*.safetensors`, `*.ckpt`, `*.dcm`, `*.zip`, `.env`, `*.token`, `hf_token*`, PDF ở gốc, `papers/`, `*.xlsx`. Nó **không** chặn `kaggle.json`, `*.dicom`, `*.IMG` nếu chúng nằm ngoài `data/`: đừng để các file đó trong repo. Đừng xóa các dòng có sẵn. Luôn chạy `git status` trước `git add`. Không dùng `git add -A` hay `git add .` khi chưa xem `git status`.

`report/main.pdf` và `docs/handoff/so_tay_ban_giao.pdf` **được** commit (`.gitignore` chỉ chặn PDF ở gốc repo). File phụ của LaTeX (`.aux`, `.log`, `.fdb_latexmk`…) bị chặn.

7.5 Ghi lại quyết định

Mọi quyết định thay đổi cách làm (đổi siêu tham số, cắt ngân sách, chọn cấu hình tốt nhất, bỏ một bộ dữ liệu…) được ghi vào `docs/decisions.md`. Mỗi mục có: **mã** (D-số), **ngày**, **quyết định**, **lý do**, **nguồn** (spec mục nào, PR nào, file kết quả nào). Không sửa mục cũ; nếu đổi ý, thêm mục mới ghi "thay thế D-x".

File đã có sẵn các quyết định của nhóm trước (D1–D27), lấy từ spec và §8.5. Ví dụ một mục mới:

```
## D28 - 2026-12-02 - Best LoRA config for E4/E5 is C3
- Decision: C3 is the best of C1-C3 (mean val_sens90 over 3 seeds: C1 0.41, C2 0.45, C3 0.52).
- Reason: spec 8.5 - choose by mean val_sens90 in val_selection.csv, never by test.
- Source: outputs/report/val_selection.csv (run 2026-12-01), PR #7.
```

(Số liệu trong ví dụ là giả. Trong file thật, viết bằng tiếng Việt như các mục có sẵn.)

8 Quy tắc bắt buộc và tiêu chí hoàn thành

8.1 Nên và không nên

✓ Nên	✗ Không được
Chọn ngưỡng, epoch, cấu hình tốt nhất **chỉ trên validation** (`val_selection.csv`).	Nhìn số trên test rồi quay lại đổi cấu hình, prompt, ngưỡng.
Dùng seed cố định 0, 1, 2 như RUNBOOK; báo mean ± sd.	Chỉ báo seed đẹp nhất.
Ghi thời gian, GPU, quyết định cắt giảm vào `outputs/budget.md`; tóm tắt sang `docs/decisions.md`.	Tự ý giảm dữ liệu/epoch mà không ghi lại.
Báo tỷ lệ `unparsed` cạnh `raw_sens` cho mọi cấu hình VLM sinh văn bản.	Bỏ qua câu trả lời không parse được, hoặc lặng lẽ coi chúng là "yes".
Dùng run 0 (nhiệt độ 0) cho bảng chính của C0; run 1–2 chỉ cho độ ổn định.	Lấy run nhiệt độ 0,7 tốt nhất cho bảng chính.
Báo CI 95% và p-value sau Holm khi so sánh.	Viết "tốt hơn" khi CI chồng nhau nhiều và p ≥ 0,05.
Báo `delta` kèm `n_t2_ok` cho E5.	Báo `flip_t1_all` như số chính.
Ghi chú PPV trên JSRT không có ý nghĩa lâm sàng (tỷ lệ nốt 62%).	Trình bày PPV trên JSRT như PPV thực tế.
Chạy 7 kiểm tra bắt buộc trước E1–E5.	Chạy thẳng E3 trên GPU khi chưa qua mục 1–3.
Chép lệnh từ RUNBOOK; thêm lệnh mới vào RUNBOOK qua PR.	Chạy lệnh tự chế rồi quên ghi.
Gọi sản phẩm là "nguyên mẫu nghiên cứu".	Viết "ngang bác sĩ" hay gợi ý dùng lâm sàng.
Nêu hạn chế: nhãn nhiễu, CI không gộp theo bệnh nhân, rò rỉ tiền huấn luyện, chia theo ảnh.	Giấu kết quả âm tính. Kết quả âm tính vẫn trả lời được Q1/Q2.

8.2 Định nghĩa "xong" cho từng thí nghiệm

Một thí nghiệm chỉ được coi là **xong** khi mọi ô dưới đây đã đánh dấu.

Chung cho mọi thí nghiệm
☐ Lệnh đã chạy được ghi lại (lệnh RUNBOOK, hoặc lệnh mới đã thêm vào RUNBOOK qua PR).
☐ Thời gian chạy, loại GPU ghi trong `outputs/budget.md`.
☐ Quyết định mới (nếu có) ghi trong `docs/decisions.md`.
☐ Không có file cấm trong commit (`git status` sạch phần dữ liệu/kết quả).

TN	Tiêu chí xong
Kiểm tra	☐ 7/7 mục đạt, ghi kết quả từng mục. ☐ Có số giây/ảnh của 2 VLM và giây/ảnh của mặt nạ phổi. ☐ Có `timing.json` của pilot và ngoại suy thời gian một lượt đầy đủ.
E0	☐ 3 manifest tồn tại. ☐ Đã xem ảnh trong `outputs/qa/` và box đúng chỗ. ☐ Có `e0_data.md`, `e0_data_edges.json`. ☐ Ghi số ca dương thật của từng split. ☐ Đã điền `tab:data-stats`.
E1	☐ `B1.jsonl`, `B2.jsonl` có đủ 3 seed × mọi ảnh val/test/test_nih/test_jsrt. ☐ 3 file `best.pt` trên Drive.
E2	☐ `C0-medgemma.jsonl`, `C0-qwen.jsonl` có run 0, 1, 2 cho mọi ảnh. ☐ Đã đọc thử `raw_text`, ghi tỷ lệ `unparsed`.
E3	☐ 9 adapter `final/` (C1–C3 × 3 seed). ☐ 9 bộ dự đoán trong `C1/C2/C3.jsonl`. ☐ Đường học (seed 0). ☐ `loss` pilot nằm trong 0,5–3.

TN	Tiêu chí xong
Cổng	☐ `make_report.py` chạy không lỗi. ☐ Cấu hình tốt nhất chọn theo trung bình `val_sens90` (chỉ C1–C3), ghi vào `decisions.md` kèm số.
E4	☐ Ít nhất 1 seed C4 huấn luyện bằng đúng cờ của cấu hình nguồn. ☐ `C4.jsonl` đủ 4 split.
E5	☐ 6 file JSON (3 mô hình × `test`, `test_jsrt`). ☐ Dùng đúng `t90` từ `thresholds.json`. ☐ Ghi loại mặt nạ phổi đã dùng.
Báo cáo	☐ Không còn ô [–]. ☐ Tóm tắt và kết luận trả lời Q1, Q2. ☐ `main.pdf` biên dịch không lỗi. ☐ Release `develop` → `main` kèm tag.

9 Hỏi đáp thường gặp

Tôi không có GPU ở nhà. Có làm được gì không?
Có. Đọc tài liệu, chạy kiểm thử CPU, chạy 3 script `prepare_*`, `draw_boxes.py`, `data_stats.py`, `make_report.py` đều không cần GPU. Chỉ huấn luyện và dự đoán VLM/CNN mới cần Colab.

Vì sao không dùng T4 miễn phí?
MedGemma cần bf16 thật. T4 chỉ giả lập bf16, và fp16 làm MedGemma tràn số. Code chủ động từ chối T4 để tránh kết quả rác.

Vì sao lấy điểm từ log-prob mà không dùng câu trả lời yes/no?
Câu trả lời yes/no chỉ cho một điểm trên đường ROC. Ta cần điểm liên tục để vẽ ROC, tính AUC và chọn ngưỡng ở specificity 0,90. Câu trả lời sinh ra vẫn được giữ để đo `raw_sens`, tỷ lệ parse lỗi và lấy box.

Spec nói 70/10/20, nhưng nếu dùng tập test chính thức VinDr-CXR thì sao?
Khi có `--official-test-csv`, code chia VinBigData Kaggle thành 85/15 (train/val) và dùng tập chính thức làm `test`. Ghi lựa chọn vào `decisions.md`.

Tôi muốn thử prompt khác. Có được không?
Trong điều kiện chính thì **không**: prompt cố định là một phần của thiết kế (spec §5.3). Nếu muốn, làm thành phân tích phụ trên nhánh `exp/`, đặt `--config` mới (không trùng C0–C4), và chọn chỉ trên val.

Kết quả C3 trên test tệ hơn C2. Tôi có đổi sang C2 cho E5 không?
Không, nếu C3 có `val_sens90` trung bình cao hơn. Cấu hình cho E4/E5 chọn theo val. Kết quả test là kết quả, không phải tiêu chí chọn.

Câu trả lời của VLM không parse được thì tính sao?
`raw_sens`/`raw_spec` coi chúng là "no". Tỷ lệ `no_on_pos`, `yes_on_neg` chỉ tính trên câu parse được. Tỷ lệ `unparsed` báo riêng. Điểm $P(\text{yes})$ (dùng cho AUC và t_{90}) không phụ thuộc parse.

Sửa parser rồi có phải chạy lại mô hình không?
Không. `make_report.py` đọc lại `raw_text` bằng parser hiện hành. Chạy lại `make_report.py` là đủ.

Một lượt LoRA chạy quá 3 giờ. Làm gì?
Cắt theo đúng thứ tự §6.10: `--neg-per-pos 2`; (chỉ Qwen) giảm độ phân giải; `--epochs 2`; C1 chỉ 1 seed. Ghi vào `budget.md`. Áp cùng mức cắt cho mọi cấu hình cần so sánh với nhau.

Colab ngắt giữa lúc huấn luyện LoRA. Có tiếp tục được không?
`train_lora.py` không có cờ chạy tiếp. Checkpoint mỗi epoch vẫn được lưu trong thư mục `checkpoint-*` bên trong `$O/lora/<cfg>-s<seed>/`, nhưng chạy lại lệnh sẽ huấn luyện từ đầu. Thêm chức năng chạy tiếp là một việc `feature/` hợp lệ (cần test và PR).

Có cần chạy C5, C6 không?
Không. Đó là phần mở rộng ngoài cam kết. Chỉ làm khi mọi thứ khác xong.

Ai quyết định khi sổ tay, RUNBOOK và spec khác nhau?
Thứ tự ưu tiên: spec > RUNBOOK > sổ tay. Báo sai lệch bằng một issue hoặc PR sửa tài liệu thấp hơn.

Dữ liệu để ở đâu cho cả nhóm cùng dùng?
Mỗi bạn chạy Colab trên Drive của mình, nên mỗi bản clone cần `data/raw/` và `data/processed/` riêng (xem `data/README.md`). Có thể chia sẻ thư mục Drive *riêng tư* trong nhóm, nhưng không chia sẻ công khai và không đưa ra ngoài nhóm (giấy phép dữ liệu).

A Bảng thuật ngữ Việt–Anh

Tiếng Việt	Tiếng Anh	Giải thích ngắn
X-quang ngực	chest X-ray (CXR)	Ảnh xám 2D chụp lồng ngực bằng tia X.
Nốt phổi	lung nodule	Đốm tròn trong phổi, ≤ 3 cm.
Khối	mass	Tổn thương phổi > 3 cm. Đồ án gộp với nốt thành lớp "Nodule/Mass".
Độ mờ	subtlety	Mức khó thấy của nốt; JSRT chấm 1 (rất khó) đến 5 (rõ).
Hộp bao	bounding box	Hình chữ nhật $[x_1, y_1, x_2, y_2]$ (pixel) quanh tổn thương.
Hợp nhất box có trọng số	weighted boxes fusion (WBF)	Gộp box của nhiều bác sĩ thành một box đồng thuận.
DICOM	DICOM	Định dạng ảnh y khoa chuẩn, chứa ảnh và thông tin máy chụp.
Bản kê dữ liệu	manifest	File CSV liệt kê mọi ảnh: đường dẫn, nhãn, box, split.
Mạng tích chập	convolutional neural network (CNN)	Mạng học đặc trưng ảnh bằng bộ lọc trượt.
Bộ phân loại	classifier	Mô hình cho một nhãn/xác suất cho cả ảnh (B1).
Bộ phát hiện	detector	Mô hình cho danh sách box kèm điểm (B2).
Mô hình ngôn ngữ lớn	large language model (LLM)	Mô hình đoán token tiếp theo trong văn bản.
Mô hình ngôn ngữ–thị giác	vision-language model (VLM)	LLM có thêm bộ mã hóa ảnh; nhận ảnh + chữ, trả lời bằng chữ.
Token	token	Mẩu từ mà LLM xử lý.
Bộ mã hóa ảnh	image encoder (ViT)	Phần biến ảnh thành dãy vector cho LLM. Đông cứng trong đồ án.
Đông cứng	frozen	Trọng số không được cập nhật khi huấn luyện.
Zero-shot	zero-shot	Dùng mô hình không qua huấn luyện thêm cho tác vụ (C0).
Tinh chỉnh	fine-tuning	Huấn luyện tiếp mô hình có sẵn trên dữ liệu mới.
Thích ứng hạng thấp	Low-Rank Adaptation (LoRA)	Học thêm BA hạng r nhỏ thay vì cập nhật cả W.

Tiếng Việt	Tiếng Anh	Giải thích ngắn
Hạng	rank (r)	Số chiều của không gian cập nhật LoRA; đồ án dùng 16.
Adapter	adapter	Bộ trọng số LoRA đã học, lưu riêng (thư mục `final/`).
Tinh chỉnh có giám sát	supervised fine-tuning (SFT)	Huấn luyện bằng cặp câu hỏi–đáp án mẫu.
Che loss	loss masking	Không tính loss trên token câu hỏi (nhãn -100).
Entropy chéo	cross-entropy	Hàm loss đo độ sai của phân phối dự đoán token.
Tích lũy gradient	gradient accumulation	Cộng gradient nhiều bước nhỏ rồi mới cập nhật một lần.
Mất cân bằng lớp	class imbalance	Một lớp hiếm hơn hẳn lớp kia (ca dương $\approx 3\%$).
Lấy mẫu lặp	oversampling	Lặp lại mẫu lớp hiếm để cân bằng (C3).
Chế độ báo bình thường	normal-mode failure	Mô hình gần như luôn nói "không có nốt": spec ≈ 1, sens ≈ 0.
Bộ phân tích câu trả lời	parser	Hàm đọc nhãn và box từ câu trả lời dạng chữ.
Nhiệt độ	temperature	Độ ngẫu nhiên khi sinh chữ; 0 = luôn chọn token xác suất cao nhất.
Ma trận nhầm lẫn	confusion matrix	Bảng 2×2: TP, FP, TN, FN.
Độ nhạy	sensitivity / recall	TP/(TP+FN): bắt được bao nhiêu ca bệnh.
Độ đặc hiệu	specificity	TN/(TN+FP): bỏ qua đúng bao nhiêu ca lành.
Giá trị dự đoán dương	positive predictive value (PPV)	TP/(TP+FP): nói "có" thì đúng bao nhiêu.
Giá trị dự đoán âm	negative predictive value (NPV)	TN/(TN+FN).
Tỷ lệ bệnh	prevalence	Tỷ lệ ca dương trong tập; ảnh hưởng mạnh tới PPV.
Ngưỡng	threshold (t_{90})	Điểm cắt; t_{90} cho specificity 0,90 trên val.
Đường ROC	ROC curve	Sens theo (1 − spec) khi thay đổi ngưỡng.
Diện tích dưới đường	AUC (AUC-ROC, AUC-PR)	Tóm tắt đường ROC/PR thành một số.
Đường FROC	free-response ROC	Sens theo số box báo nhầm mỗi ảnh.
Giao trên hợp	Intersection over Union (IoU)	Diện tích giao / diện tích hợp của hai box.
Hiệu chỉnh xác suất	calibration	Mức khớp giữa điểm dự đoán và tỷ lệ đúng thật.
Sai số hiệu chỉnh kỳ vọng	expected calibration error (ECE)	Chênh lệch trung bình giữa điểm và tỷ lệ dương theo khoảng.
Điểm Brier	Brier score	Trung bình $(s-y)^2$; bản balanced tính riêng hai lớp rồi lấy trung bình.
Khoảng tin cậy	confidence interval (CI)	Khoảng chứa giá trị thật với độ tin cậy 95%.
Lấy mẫu lại	bootstrap	Rút mẫu có hoàn lại nhiều lần để ước lượng CI.
Kiểm định McNemar	McNemar's test	So hai mô hình trên cùng ảnh qua các ca bất đồng.
Kiểm định DeLong	DeLong's test	So hai AUC trên cùng tập ảnh.
Hiệu chỉnh Holm	Holm correction	Điều chỉnh p-value khi so nhiều cặp.
Phân tầng	stratification	Tính chỉ số riêng theo nhóm (kích thước, độ mờ).
Tam phân vị	tertile	Chia thành 3 nhóm đều nhau theo một đại lượng.
Phép che vùng	occlusion test	Che một vùng ảnh và xem đáp án có đổi không (T1/T2/T3).
Tỷ lệ đổi đáp án	flip rate	Tỷ lệ ca "yes" chuyển thành "no" sau khi che.
Quá khớp	overfitting	Học thuộc train, kém trên dữ liệu mới.

Tiếng Việt	**Tiếng Anh**	**Giải thích ngắn**
Hạt giống ngẫu nhiên	random seed	Số khởi tạo bộ sinh ngẫu nhiên; cố định để tái lập.
Rò rỉ dữ liệu	data leakage	Thông tin test lọt vào lúc huấn luyện hoặc lúc chọn.
Trong / ngoài phân phối	in-distribution / out-of-distribution	Dữ liệu giống / khác nguồn dữ liệu huấn luyện.
Tập kiểm định	validation set	Dùng để chọn ngưỡng và cấu hình.
Tập kiểm tra	test set	Chỉ dùng để chấm điểm cuối, một lần.
Nhánh	branch	Dòng phát triển riêng trong Git.
Yêu cầu gộp	pull request (PR)	Đề nghị gộp một nhánh vào nhánh khác, kèm review.
Thẻ	tag	Tên gắn cố định cho một commit (ví dụ `v0.1.0-handoff`).